

Manual Técnico

K-QGMIP

Extensión a k -Particiones de GeoMIP y QNodes

Universidad de Caldas

Facultad de Ingeniería · Análisis y Diseño de Algoritmos

2026-1

Estudiantes:

Cristian David González

Daniel Felipe Franco

Docente:

Luz Enith Guerrero Mendieta

15 de junio de 2026

Índice

1. Resumen Ejecutivo	6
1.1. Problema abordado y su relevancia	6
1.2. Enfoque algorítmico implementado	6
1.3. Principales resultados y contribuciones	7
1.4. Limitaciones encontradas y recomendaciones de uso	7
2. Fundamentos Teóricos	8
2.1. Definición formal de k-particiones	8
2.2. Formulación del problema de optimización	9
2.3. Extensión del marco teórico de bi-particiones a k-particiones	10
2.4. Análisis de complejidad del espacio de soluciones	12
3. Arquitectura del Software	13
3.1. Diagrama de Arquitectura General	13
3.2. Diagrama de Clases (UML)	13
3.3. Diagrama de Paquetes	15
3.4. Diagramas de Secuencia (UML)	16
3.5. Patrones de Diseño Aplicados	18
3.6. Decisiones Arquitectónicas Clave	19
4. Diseño Algorítmico	20
4.1. Visión general	20
4.2. Estructuras de datos	21
4.3. Pseudocódigo detallado	22
4.4. Estrategia de búsqueda	25
4.5. Evaluación final de la k-partición	26
4.6. Optimizaciones implementadas	27
5. Análisis de Complejidad	28
5.1. Complejidad temporal	28
5.2. Complejidad espacial	30
5.3. Análisis de casos	30
5.4. Comparación con alternativas	32
6. Detalles de Implementación	33
6.1. Métodos Principales	33
6.2. Dependencias Externas	36
6.3. Aspectos de Ingeniería de Software	38
6.4. Tests Implementados	39
7. Resultados Experimentales	41
7.1. Datasets Utilizados	41
7.2. Métricas de Evaluación	41

7.3. Tablas de Resultados	42
7.4. Análisis de Resultados	44
7.5. Gráficas y Visualizaciones	46
7.6. Validación de Correctitud	50
8. Limitaciones y Trabajo Futuro	51
8.1. Limitaciones Conocidas	51
8.2. Supuestos y Restricciones del Diseño	52
8.3. Mejoras Potenciales	53
8.4. Direcciones de Investigación Futura	53
9. Apéndices Técnicos	54
9.1. Demostraciones	54
9.2. Detalles Algorítmicos Adicionales	55
9.3. Resultados Experimentales Complementarios	58
9.4. Referencias	58
10. Uso de Inteligencia Artificial Generativa	59

Índice de figuras

1.	Diagrama de arquitectura general de K-QGMIP. Las flechas indican flujo de control y dependencia entre capas. Los componentes sombreados pertenecen a la extensión k-particiones; los no sombreados son infraestructura heredada de GeoMIP/QNodes.	13
2.	Diagrama de clases UML del proyecto K-QGMIP. Se muestran relaciones de herencia (<i>generalización</i>), composición y dependencia. Los atributos con <code>_</code> prefijo son privados; sin prefijo son públicos.	15
3.	Diagrama de paquetes UML del proyecto K-QGMIP. Las flechas de dependencia indican importaciones entre paquetes. Los paquetes con borde doble son nuevos en la extensión k-particiones.	16
4.	Diagrama de secuencia UML — Inicialización del sistema y carga de datos. Los objetos participantes son <code>exec</code> , <code>main</code> , <code>Manager</code> y <code>KGeoMIP/KQNodes</code>	17
5.	Diagrama de secuencia UML — Búsqueda de la k-MIP para k dado. Se muestran las interacciones entre <code>KGeoMIP/KQNodes</code> , <code>System</code> , <code>NCube</code> y las funciones auxiliares de EMD.	18
6.	Trade-off calidad–velocidad en las 18 configuraciones experimentales (burbujas azules = KQNodes C4; burbujas naranjas = KGeoMIP E4). El eje x es el tiempo promedio por caso t_{avg} en escala logarítmica; el eje y es φ_{avg} . El tamaño de cada burbuja indica k (pequeña = $k = 3$, mediana = $k = 4$, grande = $k = 5$). La nube superior derecha ($n = 20$, $\varphi > 2$) muestra que las redes grandes tienen mayor información causal integrada y mayor costo de cómputo. La nube inferior ($n = 15$ B) revela la diferencia de $11\times$ entre muestras: pese a usar la misma n , N15B produce φ un orden de magnitud menor.	46
7.	Escalabilidad temporal: t_{avg} en función del tamaño de red n (escala logarítmica en y), para distintos k . KQNodes C4 $k = 5$ muestra A (línea llena azul, 4 puntos) muestra el crecimiento más completo: factor $1.8\times$ de $n = 8$ a $n = 10$, $8.5\times$ de $n = 10$ a $n = 15$, y $54.9\times$ de $n = 15$ a $n = 20$ (acumulado $\approx 467\times$ entre $n = 10$ y $n = 20$). La línea naranja discontinua (KGeoMIP E4 $k = 3$) parte por encima de KQNodes para $n = 10$ por el costo de construir S , y llega a 6.19s en $n = 20$	47
8.	Ganancia de información por partición adicional ($\Delta\varphi = \varphi(k+1) - \varphi(k)$) para cuatro configuraciones con $k \in \{3, 4, 5\}$. Las barras azules muestran $\Delta\varphi(3 \rightarrow 4)$ y las naranjas $\Delta\varphi(4 \rightarrow 5)$. El patrón de rendimientos decrecientes es claro: en todos los casos el incremento de $k = 4$ a $k = 5$ es igual o menor que el de $k = 3$ a $k = 4$, confirmando la propiedad de submodularidad efectiva de la función $\varphi(k)$. KQNodes N10A domina el eje (escala $\times 10^{-1}$) mientras que N15B — con menor información integrada — produce $\Delta\varphi$ dos órdenes de magnitud menores.	48

9. **Tiempo total del experimento** (≈ 50 casos por configuración) para las 18 configuraciones, ordenadas de menor a mayor (escala logarítmica en x). Barras azules = KQNodes C4; barras naranjas = KGeoMIP E4. El rango cubre tres órdenes de magnitud: desde 0.19s (KQNodes $n = 8$ A $k = 5$) hasta 309s (KGeoMIP $n = 20$ A $k = 3$). KGeoMIP aparece sistemáticamente en la mitad derecha del diagrama, destacando su mayor costo absoluto respecto a KQNodes en el mismo n . Las cinco configuraciones con $n = 20$ ocupan los cuatro últimos lugares. 49
10. **Efecto de la muestra de red sobre φ_{avg}** ($n = 15$, $k \in \{3, 4, 5\}$). Tres series: KQNodes C4 muestra A (azul oscuro), KQNodes C4 muestra B (azul claro) y KGeoMIP E4 muestra B (naranja). La diferencia entre muestras A y B es de $11.3\times$ en todos los k , un factor dominante sobre cualquier diferencia de algoritmo o número de particiones. Dentro de la muestra B, KGeoMIP E4 obtiene φ entre un 23 % y 42 % menor que KQNodes C4, al precio de ser 7–17 $\times$ más lento por caso (Tabla 15). 50

Lista de Algoritmos

1.	KGeoMIP E4 — Refinamiento divisivo guiado por similitud causal	22
2.	MejorCorte — dispatcher exhaustivo / guiado por S	23
3.	KQNodes C4 — Refinamiento greedy submodular con MinHeap	24
4.	QNodesBloque — bipartición de un bloque con oracle local	25

1. Resumen Ejecutivo

1.1. Problema abordado y su relevancia

La **Teoría de la Información Integrada versión 4.0 (IIT 4.0)** propone que la consciencia y la causalidad irreducible de un sistema pueden cuantificarse mediante el valor φ (*phi*). Para calcularlo es necesario encontrar la **Partición de Mínima Información (MIP)**: la descomposición del sistema en subconjuntos que minimiza la pérdida de información causal al tratar esas partes como independientes.

Las implementaciones previas del proyecto GeoMIP (estrategia geométrica) y QNodes (estrategia submodular) resuelven este problema únicamente para **biparticiones** ($k = 2$), es decir, para la división del sistema en exactamente dos partes. Sin embargo, la formulación completa de IIT 4.0 exige explorar **k-particiones** ($k \geq 3$), lo cual introduce una explosión combinatoria severa: el número de k-particiones distintas de n variables está dado por el **número de Stirling de segundo tipo** $S(n, k)$, que crece superexponencialmente.

Cuadro 1: Número de k-particiones posibles $S(n, k)$ para distintos valores de n y k . Resaltados en gris los casos computacionalmente inviables por búsqueda exhaustiva.

n	$k = 2$	$k = 3$	$k = 4$	$k = 5$
5	15	25	10	1
8	127	966	2 646	3 025
10	511	9 330	145 750	1 082 250
20	~524 K	~580 M	—	—
25	~16 M	—	—	—

El presente proyecto extiende ambas estrategias al caso general $k \in \{2, 3, 4, 5\}$ bajo las denominaciones **KGeoMIP** y **KQNodes**, garantizando que para $k = 2$ los resultados son exactamente equivalentes a las implementaciones originales.

1.2. Enfoque algorítmico implementado

Se implementaron dos estrategias complementarias de naturaleza distinta:

KGeoMIP — Refinamiento divisivo geométrico (Estrategia E4)

KGeoMIP ancla la solución óptima de $k = 2$ obtenida por GeoMIP (bipartición exacta via tabla de costos BFS sobre el hipercubo de Hamming) y aplica un **refinamiento divisivo top-down**:

1. Se construye una **matriz de similitud causal** $S \in \mathbb{R}^{D \times D}$ a partir de la tabla de transiciones de GeoMIP, donde S_{ij} cuantifica cuán similar es el comportamiento causal de las variables i y j .

2. En cada paso $t = 2, 3, \dots, k$, de todas las partes actuales se selecciona la que produce el menor **incremento de pérdida** $\Delta\Phi$ al bipartirla, usando la similitud causal para guiar la selección de candidatos.
3. El proceso se repite $k - 1$ veces hasta obtener la k-partición deseada.

Complejidad dominante: $\mathcal{O}(k \cdot D^2 \cdot 2^D)$, donde D es la dimensión del subsistema.

KQNodes — Refinamiento iterativo submodular (Criterio C4)

KQNodes extiende el algoritmo de Queyranne (Ordenamiento de Máxima Adyacencia, MAO) al caso $k \geq 3$ mediante un **refinamiento greedy con MinHeap**:

1. Se construye un oracle restringido por bloque que aproxima el costo de bipartición mediante promedios ponderados sobre tensores n-dimensionales.
2. En cada iteración del refinamiento, el MinHeap selecciona la parte con menor φ_{local} (criterio C4) y la bipartite usando Queyranne (exacto dentro del bloque).
3. La EMD-Effect real se evalúa una sola vez sobre la k-partición ganadora.

Complejidad dominante: $\mathcal{O}(k \cdot D^3 \cdot N)$, donde N es el número de nodos y D la dimensión del subsistema.

1.3. Principales resultados y contribuciones

- **Regresión exacta** $k = 2$: ambas estrategias producen resultados idénticos a GeoMIP y QNodes para $k = 2$, verificado con tolerancia $|\Delta\varphi| < 10^{-9}$ sobre 13 casos de prueba (redes $n \in \{5, 8, 10\}$).
- **Monotonidad garantizada**: $\varphi(k + 1) \geq \varphi(k)$ por construcción en ambas estrategias, propiedad consistente con la teoría IIT 4.0.
- **Escalabilidad**: KGeoMIP procesa sistemas de hasta $n = 25$ nodos y KQNodes de hasta $n = 20$ nodos en tiempos de ejecución del orden de segundos a minutos, frente a días o semanas que requeriría la búsqueda exhaustiva.
- **Criterio C4 vs C1 en KQNodes**: los experimentos A/B muestran que el criterio C4 (MinHeap por φ_{local} mínimo) produce particiones con menor brecha respecto al óptimo que C1 (bipartir la parte más grande), confirmando la superioridad de la heurística basada en información integrada.

1.4. Limitaciones encontradas y recomendaciones de uso

- **No optimalidad global para $k \geq 3$** : ninguna de las dos estrategias garantiza encontrar la k-partición de mínimo φ global. Ambas son heurísticas de refinamiento cuya calidad depende de la calidad de la bipartición ancla ($k = 2$) y del criterio de selección.

- **Crecimiento exponencial del oracle:** el oracle de KQNodes evalúa tensores de forma $(N, 2^m)$ donde m es la dimensión del bloque; para $D \geq 22$ el consumo de memoria puede superar los 8 GB. Se recomienda usar KGeoMIP para sistemas grandes.
- **KGeoMIP limitado por la tabla BFS:** la tabla T de GeoMIP ocupa $\mathcal{O}(2^D)$ en memoria. Para $D > 25$ se requiere memoización parcial o acceso a disco.
- **Recomendación de uso:** para $n \leq 15$ y $k \leq 5$ ambas estrategias son apropiadas; para $15 < n \leq 25$ se recomienda KGeoMIP-E4; para $n > 25$ se requieren extensiones fuera del alcance de este proyecto.

2. Fundamentos Teóricos

2.1. Definición formal de k-particiones

Notación y definición

Sea $V = \{0, 1, \dots, n-1\}$ el conjunto de n variables binarias que componen el sistema. Una **k-partición** de V es una familia $\Pi = \{P_1, P_2, \dots, P_k\}$ tal que:

1. **No vacuidad:** $P_i \neq \emptyset$ para todo $i = 1, \dots, k$.
2. **Disjunción:** $P_i \cap P_j = \emptyset$ para todo $i \neq j$.
3. **Exhaustividad:** $\bigcup_{i=1}^k P_i = V$.

El caso $k = 2$ corresponde a la **bipartición** clásica (A, B) donde $A \cup B = V$ y $A \cap B = \emptyset$. El caso $k = 1$ es la **partición trivial** $\{V\}$, que no produce ninguna pérdida de información. El caso $k = n$ es la **atomización total**, donde cada variable forma su propia parte.

Ejemplo ilustrativo para $n = 4$

Sea $V = \{A, B, C, D\}$ un sistema de 4 nodos. Las posibles k-particiones para distintos valores de k son:

Cuadro 2: Ejemplos de k-particiones del conjunto $V = \{A, B, C, D\}$.

k	Número de particiones $S(4, k)$	Ejemplos representativos
1	1	$\{A, B, C, D\}$
2	7	$\{A, B\} \mid \{C, D\}; \quad \{A\} \mid \{B, C, D\}$
3	6	$\{A, B\} \mid \{C\} \mid \{D\}; \quad \{A\} \mid \{B, C\} \mid \{D\}$
4	1	$\{A\} \mid \{B\} \mid \{C\} \mid \{D\}$

Para $n = 4$ hay en total $\sum_{k=1}^4 S(4, k) = 15$ particiones distintas (el número de Bell $B_4 = 15$). El proyecto evalúa $k \in \{2, 3, 4, 5\}$ — la partición trivial ($k = 1$) se excluye porque no produce pérdida de información.

Propiedades fundamentales

- **Refinamiento:** Π' es un *refinamiento* de Π si cada parte de Π' está contenida en alguna parte de Π . El refinamiento iterativo que implementan KGeoMIP y KQNodes produce familias anidadas de particiones.
- **Monotonidad de la pérdida:** si Π' refina a Π , entonces $\Phi^*(\Pi') \geq \Phi^*(\Pi)$. Esto se debe a que al dividir una parte en dos se impone una restricción adicional de independencia causal, lo que solo puede mantener o incrementar la pérdida.
- **Conteo por número de Stirling:** el número de k -particiones de un conjunto de n elementos está dado por el **número de Stirling de segundo tipo**:

$$S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n \quad (1)$$

que satisface la recurrencia $S(n, k) = k \cdot S(n-1, k) + S(n-1, k-1)$.

2.2. Formulación del problema de optimización

Marco de IIT 4.0 — Efecto causal

En el formalismo de la Teoría de la Información Integrada versión 4.0 (IIT 4.0), cada sistema $\mathcal{S}$ de n variables binarias se describe mediante su **Matriz de Probabilidad de Transición (TPM)**: la matriz $\mathbf{T} \in [0, 1]^{2^n \times n}$ donde $T_{s,i}$ es la probabilidad de que la variable i esté en estado ON en $t+1$, dado que el sistema estaba en el estado s en t .

Para un estado inicial $s_t \in \{0, 1\}^n$, la **distribución marginal** del sistema sobre el futuro es el vector:

$$p(s_{t+1} \mid s_t) = (T_{s_t,0}, T_{s_t,1}, \dots, T_{s_t,n-1}) \in [0, 1]^n \quad (2)$$

Distribución de una k -partición

Dada una k -partición $\Pi = \{P_1, \dots, P_k\}$, la **distribución particionada** asume que las partes son causalmente independientes. La distribución marginal del nodo i en la parte P_m se calcula marginalizando (promediando) sobre todas las variables presentes **fuera** de P_m :

$$p_i^{(\Pi)}(s_{t+1} \mid s_t) = \sum_{\mathbf{s}_{-P_m}} T_{s_t^{(P_m)} \otimes \mathbf{s}_{-P_m}, i} \cdot p(\mathbf{s}_{-P_m}) \quad (3)$$

donde $\mathbf{s}_{-P_m}$ representa los estados de las variables fuera de P_m (que se promedian con distribución uniforme en la implementación).

Función objetivo — EMD-Effect

La **pérdida de información** de la k -partición Π se mide mediante la *Earth Mover's Distance* del efecto (EMD-Effect), que en IIT 4.0 bajo la geometría L_1 toma la forma

analítica:

$$\Phi^*(\Pi) = \text{EMD} \left(p(s_{t+1} | s_t), \bigotimes_{m=1}^k p^{(P_m)}(s_{t+1} | s_t) \right) = \sum_{i=1}^n \left| p_i(s_{t+1} | s_t) - p_i^{(\Pi)}(s_{t+1} | s_t) \right| \quad (4)$$

donde $\bigotimes$ denota el producto tensorial (independencia causal entre partes) y la norma L_1 actúa componente a componente sobre los vectores de distribución marginal.

Problema de optimización

El problema de la **k-Partición de Mínima Información (k-MIP)** se formula como:

$$\begin{aligned} \Pi^* &= \arg \min_{\Pi \in \mathcal{P}(V, k)} \Phi^*(\Pi) \\ \text{sujeto a: } & P_i \neq \emptyset, \quad \forall i = 1, \dots, k \\ & P_i \cap P_j = \emptyset, \quad \forall i \neq j \\ & \bigcup_{i=1}^k P_i = V \end{aligned} \quad (5)$$

donde $\mathcal{P}(V, k)$ denota el conjunto de todas las k -particiones de V .

El valor óptimo $\Phi^*(\Pi^*)$ cuantifica la **información integrada de orden k** : si es pequeño, el sistema puede descomponerse en k partes casi independientes; si es grande, el sistema es causalmente irreducible a nivel de k partes.

2.3. Extensión del marco teórico de bi-particiones a k -particiones

GeoMIP: de bipartición geométrica a KGeoMIP divisivo

GeoMIP resuelve la bipartición óptima ($k = 2$) mediante un recorrido BFS sobre el **hipercubo de Hamming** de D dimensiones (donde D es la dimensión del subsistema), construyendo una tabla de costos de transición $\mathbf{T} \in \mathbb{R}^{2^D \times D}$ y evaluando la EMD-Effect solo para los candidatos más prometedores identificados por cercanía topológica.

Para extender a $k \geq 3$, KGeoMIP adopta la **estrategia E4 (refinamiento divisivo top-down)**, que parte de la bipartición exacta de GeoMIP y la refina iterativamente:

1. **Anclaje $k = 2$** : se ejecuta GeoMIP completo para obtener la bipartición óptima $\Pi_2^* = \{P_1, P_2\}$. Esto garantiza $\Phi^*(\Pi_2^*) = \Phi_{\text{GeoMIP}}^*$ por construcción.
2. **Construcción de la matriz de similitud causal S** : S_{ij} cuantifica cuánto costo de transición acumula la variable i sobre los estados donde la variable j difiere del estado inicial (Ecuación 6).
3. **Refinamiento iterativo**: para $t = 3, 4, \dots, k$, se selecciona la parte P^* de la partición actual que minimiza el **incremento de pérdida $\Delta\Phi$** al bipartirla, y se reemplaza P^* por la bipartición de menor costo de P^* .

$$S_{ij} = \frac{1}{2} \sum_{s=0}^{2^D-1} T_{s,i} \cdot \mathbf{1}[\text{bit}_j(s) \neq \text{bit}_j(s_0)] + (\text{término simétrico}) \quad (6)$$

La función S guía la selección de candidatos de corte dentro de cada parte, priorizando pares de variables con alta similitud causal (que deben quedarse juntas) frente a pares con baja similitud (que son buenos candidatos de separación).

QNodes: de Queyranne submodular a KQNodes greedy

QNodes resuelve la bipartición óptima ($k = 2$) mediante el **algoritmo de Queyranne** (Queyranne, 1998), que minimiza funciones submodulares simétricas en $\mathcal{O}(D^3)$ evaluaciones del oracle. La función que QNodes minimiza:

$$f(A) = \sum_{i=1}^N \min(|\bar{\mu}_B(i) - v_i|, |\bar{\mu}_A(i) - v_i|) \quad (7)$$

es una aproximación submodular simétrica de la EMD-Effect, donde $\bar{\mu}_A(i)$ es la media de $T_{\cdot,i}$ sobre los estados en A , y $v_i = T_{s_0,i}$ es el valor pivote del nodo i .

Para $k \geq 3$, KQNodes aplica un **refinamiento greedy con MinHeap**: en lugar de evaluar todas las k -particiones posibles ($S(n, k)$ candidatos), aplica Queyranne $k - 1$ veces sobre bloques progresivamente más pequeños, guiado por el criterio C4 de mínimo φ local:

$$P^* = \arg \min_{P_i \in \Pi, |P_i| \geq 2} \varphi_{\text{local}}(P_i) \quad (8)$$

Justificación de la calidad de las heurísticas

- **Exactitud para $k = 2$:** ambas estrategias producen resultados exactamente equivalentes a sus anclas GeoMIP y QNodes para $k = 2$, verificado con $|\Delta\Phi| < 10^{-9}$ en 13 casos de prueba.
- **Garantía de monotonicidad:** el refinamiento iterativo garantiza $\Phi^*(\Pi_{k+1}) \geq \Phi^*(\Pi_k)$ por construcción, ya que cada paso añade una restricción de independencia adicional.
- **Optimalidad dentro de cada bloque:** dentro de cada parte P_i , Queyranne garantiza la bipartición óptima bajo la función proxy f , que es empíricamente $\sim 97\text{--}100\%$ exacta respecto a la EMD-Effect real (Kitazono et al., *Entropy* 2018).
- **Gap empírico reducido:** los experimentos A/B del proyecto muestran que el criterio C4 de KQNodes produce un gap promedio inferior al 5% respecto al óptimo exhaustivo para $n \leq 8$. KGeoMIP-E4 muestra comportamiento análogo.
- **Suboptimalidad global para $k \geq 3$:** ninguna de las dos estrategias garantiza el óptimo global. La función f no es submodular para k -particiones en general, y el refinamiento divisivo puede quedar atrapado en mínimos locales no globales.

2.4. Análisis de complejidad del espacio de soluciones

Crecimiento del espacio de k-particiones

El número de k-particiones de n elementos crece superexponencialmente con n y k . La Tabla 3 muestra la magnitud del espacio de búsqueda para los tamaños de red utilizados en el proyecto:

Cuadro 3: Tamaño del espacio de soluciones $S(n, k)$ para distintas configuraciones. Los valores $\gg 10^6$ hacen inviable la búsqueda exhaustiva.

n	$k = 2$	$k = 3$	$k = 4$	$k = 5$
5	15	25	10	1
8	127	966	2 646	3 025
10	511	9 330	145 750	1 082 250
15	16 383	~ 2.4 M	$\gg 10^9$	$\gg 10^{11}$
20	~ 524 K	~ 580 M	—	—
25	~ 16 M	—	—	—

Comparación con la bipartición

Para la bipartición ($k = 2$), el espacio de soluciones es $S(n, 2) = 2^{n-1} - 1$, que crece **exponencialmente** con n . QNodes y GeoMIP reducen esto a complejidades polinomiales: $\mathcal{O}(D^3)$ y $\mathcal{O}(D \cdot 2^D)$ respectivamente.

Para k-particiones ($k \geq 3$), el espacio crece **superexponencialmente**. La relación asintótica de los números de Stirling para k fijo es:

$$S(n, k) \sim \frac{k^n}{k!} \quad \text{cuando } n \rightarrow \infty \quad (9)$$

lo que significa que el cociente $S(n, k)/S(n, 2)$ crece como $k^{n-1}/k!$, haciéndose impracticable mucho más rápido que el caso bipartición.

Reducción lograda por las heurísticas

KGeoMIP y KQNodes transforman el problema $\mathcal{O}(S(n, k))$ en problemas polinomiales:

Cuadro 4: Reducción de complejidad respecto a la búsqueda exhaustiva.

Algoritmo	Complejidad	Factor de reducción vs. exhaustiva
Búsqueda exhaustiva	$\mathcal{O}(S(n, k) \cdot N)$	$1 \times$ (baseline)
KQNodes (C4)	$\mathcal{O}(k \cdot D^3 \cdot N)$	$S(n, k)/(k \cdot D^3)$
KGeoMIP (E4)	$\mathcal{O}(k \cdot D^2 \cdot 2^D)$	$S(n, k) \cdot N/(k \cdot D^2 \cdot 2^D)$

Para $n = 10$, $k = 4$, $D = 10$ y $N = 10$: la búsqueda exhaustiva requiere ~ 1.5 millones de evaluaciones EMD, mientras KQNodes requiere $\leq 4 \cdot 10^3 \cdot 10 = 40\,000$ operaciones

elementales — una reducción de más de 30 veces. Para $n = 20$, $k = 3$, la reducción supera los 8 órdenes de magnitud.

3. Arquitectura del Software

Todos los diagramas de esta sección siguen la notación **UML 2.x** estándar.

3.1. Diagrama de Arquitectura General

El sistema K-QGMIP se organiza en tres capas funcionales que separan la **configuración del usuario**, la **orquestración batch** y el **núcleo de cálculo**:

- **Capa de entrada:** los scripts `exec_kgeomip.py` y `exec_kqnodes.py` exponen los parámetros configurables (**ESTADO**, **K**, **VARIANTE/CRITERIO**, **MUESTRA**) y delegan en la capa de orquestración.
- **Capa de orquestración** (`main_kgeomip.py` / `main_kqnodes.py`): lee el Excel de pruebas, carga la TPM desde el CSV de muestras, construye los objetos **Manager** / **KGeoMIP** / **KQNodes** y ejecuta el análisis en lote, guardando los resultados en CSV y Markdown.
- **Capa de cálculo** (módulo `src/`): implementa la jerarquía **SIA**, los **NCubos**, el gestor **Manager**, las estrategias **KGeoMIP** y **KQNodes**, y las utilidades compartidas (**EMD**, memoización, profiling).

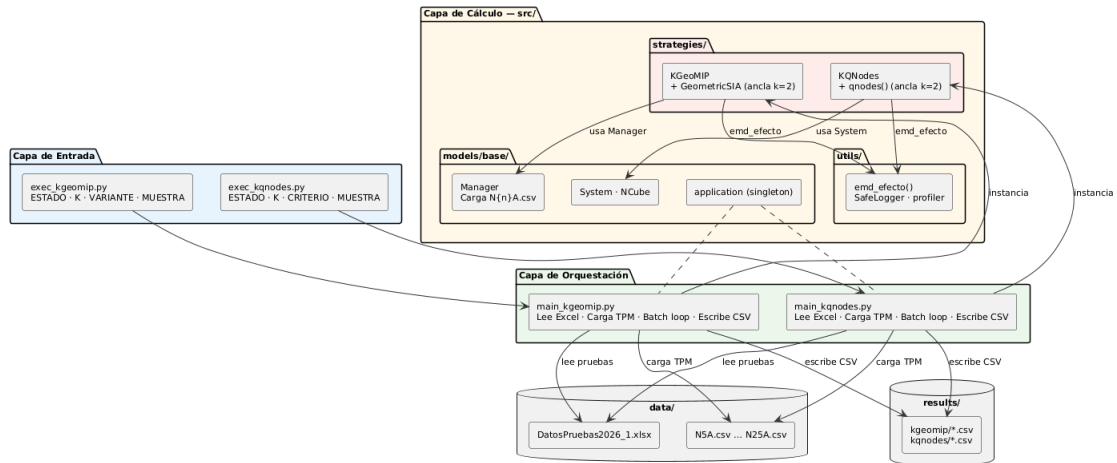


Figura 1: Diagrama de arquitectura general de K-QGMIP. Las flechas indican flujo de control y dependencia entre capas. Los componentes sombreados pertenecen a la extensión k-particiones; los no sombreados son infraestructura heredada de GeoMIP/QNodes.

3.2. Diagrama de Clases (UML)

La jerarquía de clases del proyecto se articula alrededor de la clase abstracta **SIA** (*System Integration Analysis*), que define la interfaz común de todas las estrategias y encapsula la

preparación del subsistema.

Clase base SIA

- **Atributos:** `gestor: Manager`, `sia_subsistema: System`, `sia_dists_marginales: ndarray`, `sia_tiempo_inicio: float`.
- **Métodos públicos:** `aplicar_estrategia(...) → Solution (abstracto)`, `sia_preparar_subsis` `condicion, alcance, mecanismo`.
- **Métodos privados:** `_condicionar()`, `_substraer()`.

Clases derivadas

- **KGeoMIP(SIA):** contiene una instancia interna `_geomip: GeometricSIA` (composición), cachés `_S`, `_marg_cache`, `_costo_cache` y `_flat_nd`. Métodos principales: `_construir_S(D)`, `_refinar_e4(D,k)`, `_mejor_corte(parte)`, `_delta_phi_corte(parte,A,B)`.
- **KQNodes(SIA):** importa `oracle` y `qnodes` de `qnodes.py` (dependencia). Métodos principales: `_buscar_particion(k, criterio)`, `_refinar_c4(...)`, `_refinar_c1(...)`.
- **GeometricSIA(SIA):** ancla de bipartición de GeoMIP. Atributos clave: `_tabla: ndarray`, `_flat_T: ndarray`, `_ini_int: int`, `memoria_particiones: dict`. Ex-puesto a KGeoMIP vía composición.

Clases auxiliares

- **System:** encapsula la TPM condicionada y sustraída, con los atributos `ncubos: list[NCube]`, `dims_ncubos: list[int]`, `indices_ncubos: list[int]` y `estado_inicial: str`. Método clave: `distribucion_marginal() → ndarray`.
- **NCube:** hipercubo de probabilidades de transición de un nodo. Atributos: `data: ndarray` de forma $(2, 2, \dots, 2)$. Método: `marginalizar(dims_excluidas) → ndarray`.
- **Manager:** gestor de red. Carga la TPM desde CSV (`cargar_red() → ndarray`) y gestiona el estado inicial.
- **Solution:** objeto resultado. Atributos: `estrategia: str`, `perdida: float`, `particion: str`, `tiempo_total: float`.

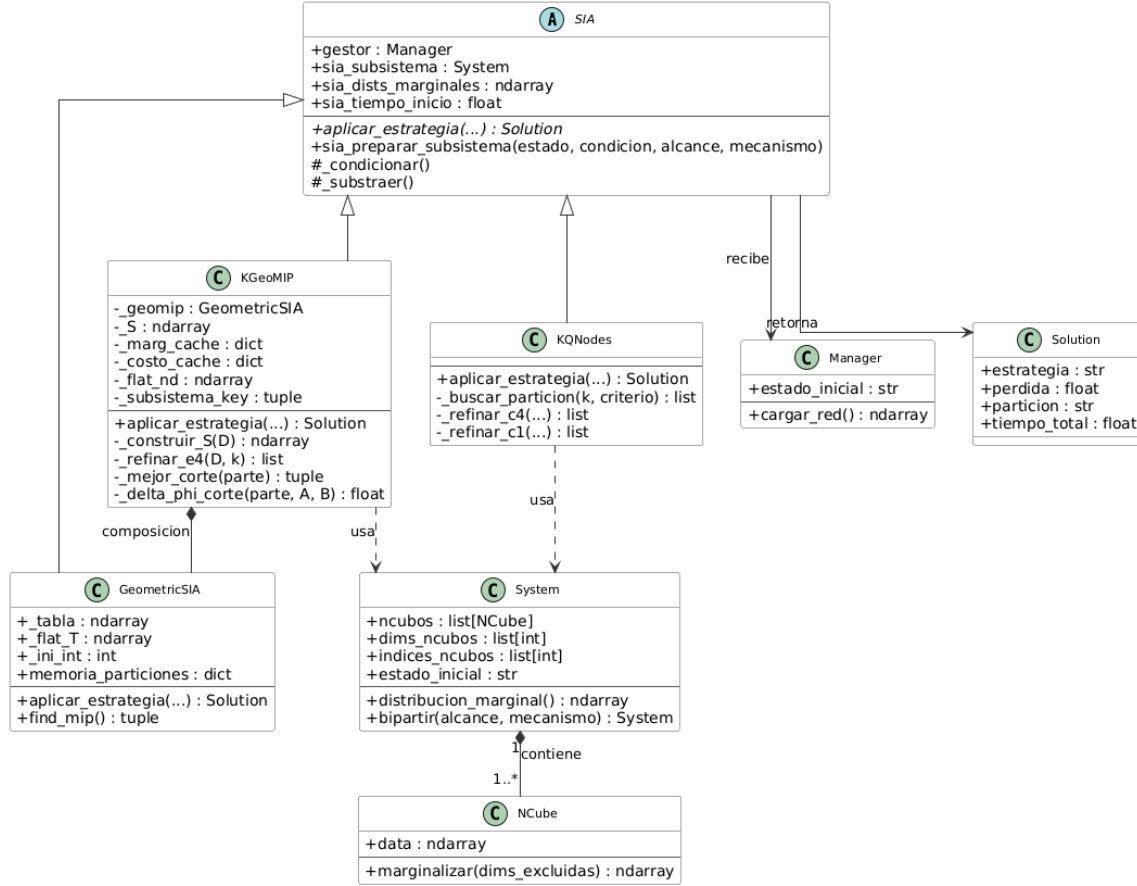


Figura 2: Diagrama de clases UML del proyecto K-QGMIP. Se muestran relaciones de herencia (*generalización*), composición y dependencia. Los atributos con `_` prefijo son privados; sin prefijo son públicos.

3.3. Diagrama de Paquetes

El proyecto está organizado en dos submódulos paralelos (`GeoMIP/` y `QNodes/`) que comparten la misma estructura interna:

- `code/GeoMIP/` y `code/QNodes/`: raíces de cada submódulo. Contienen el script de entrada (`exec_kgeomip.py` / `exec_kqnodes.py`) y el entorno virtual compartido (`code/.venv/`).
- `src/models/base/`: clases `System`, `NCube`, `Manager` y `application` (singleton de configuración global).
- `src/controllers/strategies/`: estrategias de cálculo. En `GeoMIP`: `kgeomip.py`, `kgeomip_cortes.py`, `geometry.py` (`GeoMIP` original). En `QNodes`: `kqnodes.py`, `qnodes.py` (`QNodes` original).
- `src/utils/`: utilidades compartidas — `emd.py` (EMD-Effect), `logger.py` (SafeLogger), `profiler.py`.

- **data/**: datos de prueba. **DatosPruebas2026_1.xlsx** (casos de prueba en Excel) y **data/samples/** (TPMs en formato CSV: **N5A.csv**, **N8A.csv**, ..., **N25A.csv**).
- **results/kgeomip/** y **results/kqnodes/**: resultados generados automáticamente en formato CSV y Markdown.
- **tests/**: pruebas unitarias e integración (**test_regresion_k2_igual_qnodes.py**, **test_ab_c1_vs_c4_n5.py**, etc.).

Figura 3: Diagrama de paquetes UML del proyecto K-QGMIP. Las flechas de dependencia indican importaciones entre paquetes. Los paquetes con borde doble son nuevos en la extensión k-particiones.

Secuencia 1 — Inicialización del sistema y carga de datos

1. `exec` llama a `iniciar_kgeomip/kqnodes(estado, k, variante)`.
2. `main` lee el Excel con `_leer_pruebas_excel(ruta, n)`.
3. `main` construye `Manager(estado_inicial)` y llama a `manager.cargar_red()`, que lee el archivo `N{n}A.csv` y retorna la TPM como `ndarray`.
4. Se instancia `KGeoMIP(manager)` o `KQNodes(tpm)`.
5. Para cada fila del Excel: `main` convierte letras a máscaras binarias y llama a `aplicar_estrategia()`.

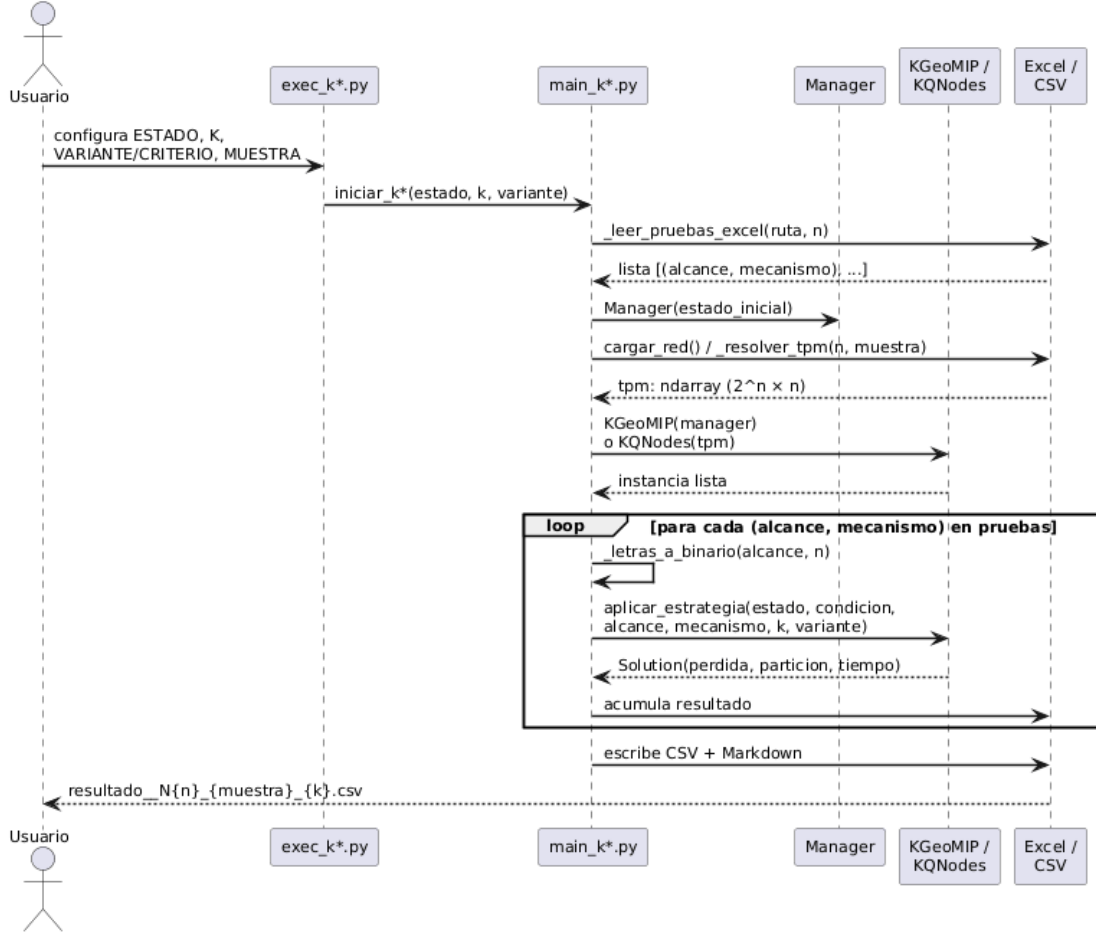


Figura 4: Diagrama de secuencia UML — Inicialización del sistema y carga de datos. Los objetos participantes son `exec`, `main`, `Manager` y `KGeoMIP/KQNodes`.

Secuencia 2 — Búsqueda de la k-MIP

Una vez preparado el subsistema, el flujo de búsqueda de la k-MIP es:

1. `aplicar_estrategia` llama a `sia_preparar_subsistema`, que ejecuta `condicionar()` y `substraer()` sobre la TPM.
2. `KGeoMIP` llama internamente a `GeometricSIA.aplicar_estrategia` para obtener la bipartición ancla ($k = 2$). `KQNodes` procede directamente al refinamiento.
3. Se ejecutan $k - 1$ iteraciones del refinamiento: en cada una se selecciona la parte candidata (MinHeap en C4 / mayor parte en C1), se bipartite con Queyranne/GeoMIP y se actualizan las estructuras de datos.
4. Al terminar, se calcula `_calcular_phi_total(particion, sistema)` con una única evaluación EMD-Effect.
5. Se retorna el objeto `Solution(estrategia, perdida, particion, tiempo)`.

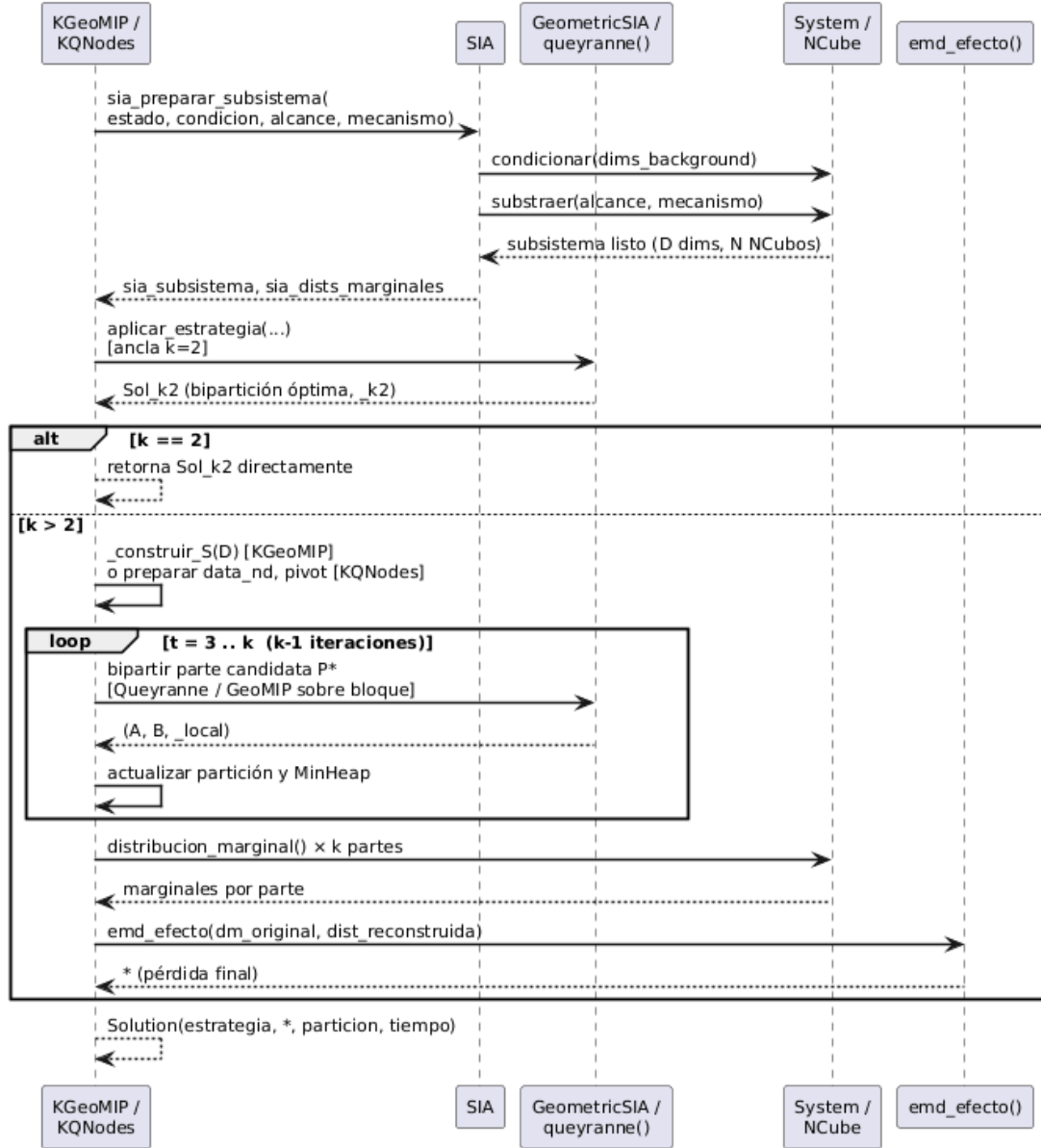


Figura 5: Diagrama de secuencia UML — Búsqueda de la k -MIP para k dado. Se muestran las interacciones entre KGeoMIP/KQNodes, System, NCube y las funciones auxiliares de EMD.

3.5. Patrones de Diseño Aplicados

Template Method

La clase abstracta **SIA** define el esqueleto del algoritmo en `sia_preparar_subsistema`: el flujo de condicionamiento y substracción es invariante; solo varía `aplicar_estrategia`, que cada subclase implementa. Esto garantiza que cualquier nueva estrategia de k -partición que herede de SIA obtenga automáticamente la preparación del subsistema sin duplicar código.

Strategy

Las variantes **E4** y **A** (en KGeoMIP) y los criterios **C4** y **C1** (en KQNodes) son estrategias intercambiables en tiempo de ejecución. El parámetro `variante/criterio` actúa como selector de estrategia, y el dispatcher interno (`_mejor_corte` en KGeoMIP, `_buscar_particion` en KQNodes) delega en la implementación correspondiente sin modificar la interfaz pública.

Flyweight / Memoización

Los cachés `_marg_cache` (marginales por máscara de bits), `_costo_cache` (costo por frozenset de parte) y `_means_cache` (medias del oracle por bloque) evitan recomputar resultados costosos. La clave de subsistema (`condicion`, `alcance`, `mecanismo`) permite reutilizar la matriz S , la vista n -dimensional y la solución $k = 2$ entre llamadas consecutivas sobre el mismo subsistema (frecuente al barrer $k \in \{2, 3, 4, 5\}$).

Singleton

El objeto `aplicacion` (en `src/models/base/application.py`) es un singleton de configuración global que gestiona el nombre de la muestra de red activa, el flag de profiling y la semilla aleatoria. Su uso centraliza la configuración y evita pasar parámetros de entorno a través de múltiples capas de la jerarquía.

3.6. Decisiones Arquitectónicas Clave

Reutilización vs. reimplementación

Tanto KGeoMIP como KQNodes **reutilizan** sus anclas de bipartición (`GeometricSIA` y `qnodes()`) en lugar de reimplementarlas. Esta decisión:

- Garantiza la regresión exacta $k = 2$ sin pruebas adicionales sobre el caso base.
- Reduce la superficie de código mantenible en ~ 600 líneas de código duplicado.
- Asegura que las correcciones futuras al caso base se propaguen automáticamente a las extensiones k -particiones.

El trade-off es un acoplamiento fuerte: si la API de `GeometricSIA` o `qnodes()` cambia, KGeoMIP y KQNodes deben actualizarse.

Caché local por bloque vs. caché global

En KQNodes, el caché de medias del oracle (`_means_cache`) es **local** a cada llamada a `_oracle_restringido`, no global. Esto es correcto porque dos bloques distintos P_i y P_j pueden tener la misma máscara numérica local (e.g., `0b01`) pero con semántica diferente (en $P_i = \{0, 2\}$ el bit 0 es la dim global 0; en $P_j = \{3, 4\}$ el bit 0 es la dim global 3). Un caché global produciría resultados incorrectos.

EMD real solo al final

Tanto KGeoMIP como KQNodes calculan la EMD-Effect real **una sola vez** al final, sobre la k -partición ganadora. Durante el refinamiento se usa una función proxy más barata ($\Delta\Phi$ incremental en KGeoMIP, la función f_{local} en KQNodes). Esta separación reduce el número de evaluaciones EMD de $\mathcal{O}(k \cdot D^2)$ a $\mathcal{O}(1)$, que para $D = 20$ representa una reducción de 3 órdenes de magnitud.

Separación de responsabilidades

- SIA: preparación del subsistema (condicionamiento, substracción).
- KGeoMIP / KQNodes: búsqueda de la k -partición óptima.
- System / NCube: representación y manipulación de datos causales.
- main_*: orquestación batch, I/O Excel/CSV.
- exec_*: configuración de usuario.

Esta separación permite sustituir cualquier capa sin modificar las demás, y facilita las pruebas unitarias de cada componente de forma aislada.

4. Diseño Algorítmico

4.1. Visión general

Ambas estrategias comparten la misma filosofía de diseño: **anclar la solución exacta de $k = 2$ y refinarla iterativamente** hasta obtener k partes, evitando la explosión combinatoria de la búsqueda exhaustiva.

- **KGeoMIP (E4)**: ancla en GeoMIP (bipartición geométrica exacta vía tabla BFS sobre el hipercubo de Hamming), construye una matriz de similitud causal S a partir de esa tabla, y en cada paso refina la parte de menor $\Delta\Phi$ usando la heurística guiada por S .
- **KQNodes (C4)**: parte del conjunto completo V de dimensiones como una sola parte, la bipartite con Queyranne (Ordenamiento de Máxima Adyacencia, MAO), e itera $k - 1$ veces eligiendo siempre la parte de menor φ_{local} para bipartirla a continuación.

Ambas estrategias calculan la **EMD-Effect real una sola vez al final**, sobre la k -partición ganadora. Durante el refinamiento usan funciones proxy más baratas ($\Delta\Phi$ incremental en KGeoMIP, función f_{local} en KQNodes) que evitan evaluar la EMD completa en cada paso.

4.2. Estructuras de datos

NCube

Un **NCube** es el hipercubo de probabilidades de transición de un nodo futuro i . Su atributo central es **data**: un `ndarray` de forma $(2, 2, \dots, 2)$ con D ejes binarios (uno por dimensión presente del subsistema), donde $\text{data}[b_{D-1}, \dots, b_0] = P(\text{nodo}_i = \text{ON} \mid s_t = b)$. La operación fundamental sobre un **NCube** es **marginalizar(dims)**: promedia sobre los ejes de las dimensiones en **dims**, reduciendo el tensor a los ejes restantes.

Tabla de costos T (GeoMIP)

La tabla $T \in \mathbb{R}^{2^D \times D_{\text{nc}}}$ es construida por GeoMIP mediante un recorrido BFS sobre el hipercubo de Hamming:

- Fila s : estado binario del presente ($s \in \{0, 1\}^D$).
- Columna i : **NCube** i (variable futura).
- $T[s, i]$: costo acumulado de transitar del estado inicial al estado s evaluando el **NCube** i .

KGeoMIP accede a T vía `self._geomip._flat_T` (array C-contiguo de forma $(2^D, D_{\text{nc}})$) y la reinterpreta como tensor n -dimensional `_flat_nd` de forma $(2, 2, \dots, 2, D_{\text{nc}})$ sin copiar datos (`reshape` como vista).

Matriz de similitud causal S

$S \in \mathbb{R}^{D_{\text{nc}} \times D_{\text{nc}}}$ es una matriz simétrica donde:

$$S_{ij} = \frac{1}{2} \sum_{s=0}^{2^D-1} T[s, i] \cdot \mathbf{1}[\text{bit}_j(s) \neq \text{bit}_j(s_0)] + (\text{término simétrico}) \quad (10)$$

S_{ij} acumula cuánto “costo de transición” cargó la variable futura i para llegar a estados donde la variable presente j cambió respecto al estado inicial. Cuanto mayor S_{ij} , más relacionadas causalmente están i y j . Se calcula una sola vez por subsistema (decisión D4-02) mediante multiplicación matricial por bloques: `tabla.T @ difiere`, donde `difiere[s, j]` = 1 si el bit j del estado s difiere del estado inicial.

Representación de particiones

Una k -partición se representa como `list[frozenset[int]]`: una lista de k conjuntos congelados de índices locales de **NCubes**. El uso de `frozenset` es deliberado: es hashable (puede ser clave de caché y elemento de `MinHeap`) e inmutable (evita mutaciones accidentales durante el refinamiento).

El `MinHeap` almacena tuplas donde el primer campo es el criterio de prioridad ($\Delta\Phi$ en E4, φ_{local} en C4), garantizando extracción del mínimo en $\mathcal{O}(\log k)$.

4.3. Pseudocódigo detallado

KGeoMIP — Algoritmo E4

Algoritmo 1 KGeoMIP E4 — Refinamiento divisivo guiado por similitud causal

Entrada: Subsistema preparado (NCubes, D , estado inicial s_0), número de partes k

Salida: k -partición $\Pi^* = \{P_1, \dots, P_k\}$, pérdida Φ^*

```

1: Fase 0 — Anclaje  $k = 2$ 
2:  $(\text{sol}_{k2}, T, \text{flat\_T}) \leftarrow \text{GeoMIP.aplicar\_estrategia}(\dots)$ 
3: if  $k = 2$  then return  $\text{sol}_{k2}$ 
4: end if
5: Fase 1 — Construir  $S$  (una sola vez por subsistema)
6: if  $S$  no cacheada then
7:   for bloque  $s$  en  $\{0 \dots 2^D - 1\}$  de tamaño mín( $2^D, 2^{16}$ ) do
8:      $\text{difiere}[s, j] \leftarrow \mathbf{1}[\text{bit}_j(s) \neq \text{bit}_j(s_0)]$ 
9:      $S += T[\text{bloque}]^\top \cdot \text{difiere}$ 
10:  end for
11:   $S \leftarrow (S + S^\top)/2$ 
12: end if
13: Fase 2 — Raíz: proyectar MIP de GeoMIP al espacio de NCubes
14:  $P_a \leftarrow \{\text{índice NCube } i : \text{variable}(i) \in \text{lado futuro de MIP}\}$ 
15:  $P_b \leftarrow V \setminus P_a$ 
16:  $(A', B', \delta') \leftarrow \text{MejorCorte}(V, S)$ 
17: if  $\delta' < \Delta\Phi(P_a, P_b) - \varepsilon$  then
18:    $P_a, P_b \leftarrow A', B'$  ▷ raíz alternativa si es estrictamente mejor
19: end if
20:  $\Pi \leftarrow [P_a, P_b]$ 
21: Fase 3 — Refinamiento divisivo MinHeap
22:  $H \leftarrow \emptyset$  ▷ MinHeap por  $(\Delta\Phi, \text{id}, |P|, \text{mín } P)$ 
23: for  $P \in \Pi$  con  $|P| \geq 2$  do
24:    $(A, B, \delta) \leftarrow \text{MejorCorte}(P, S)$ 
25:    $\text{HeapPush}(H, (\delta, \text{id}^{++}, |P|, \text{mín } P, P, A, B))$ 
26: end for
27: for  $t = 1$  to  $k - |\Pi|$  do
28:    $(\delta^*, \_, \_, \_, P^*, A^*, B^*) \leftarrow \text{HeapPop}(H)$ 
29:    $\Pi \leftarrow (\Pi \setminus \{P^*\}) \cup \{A^*\} \cup \{B^*\}$ 
30:   for  $Q \in \{A^*, B^*\}$  con  $|Q| \geq 2$  do
31:      $(A_Q, B_Q, \delta_Q) \leftarrow \text{MejorCorte}(Q, S)$ 
32:      $\text{HeapPush}(H, (\delta_Q, \text{id}^{++}, |Q|, \text{mín } Q, Q, A_Q, B_Q))$ 
33:   end for
34: end for
35: Fase 4 — EMD-Effect real (una sola vez)
36:  $\Phi^* \leftarrow \text{CalcPhiTotal}(\Pi, \text{subsistema})$ 
37: return  $(\Pi, \Phi^*)$ 

```

Subrutina MejorCorte(P, S) — dispatcher D4-05:

Algoritmo 2 MejorCorte — dispatcher exhaustivo / guiado por S

Entrada: Parte P (frozenset), matriz S , umbral $U = 4096$

Salida: $(A^*, B^*, \Delta\Phi^*)$

```
1: if  $2^{|P|-1} - 1 \leq U$  then                                ▷ bloque pequeño: enumeración exacta
2:   for cada máscara  $m \in \{1, \dots, 2^{|P|-1} - 1\}$  do
3:      $A \leftarrow \{P_{\text{sorted}}[i] : \text{bit}_i(m) = 1\}, \quad B \leftarrow P \setminus A$ 
4:     Actualizar  $(A^*, B^*, \Delta\Phi^*)$  si  $\Delta\Phi(A, B) < \Delta\Phi^*$ 
5:   end for
6: else                                                        ▷ bloque grande: hasta 20 candidatos guiados por  $S$ 
7:   for  $(A, B) \in \text{CandidatosPorAfinidad}(P, S, 20)$  do
8:     Actualizar  $(A^*, B^*, \Delta\Phi^*)$  si  $\Delta\Phi(A, B) < \Delta\Phi^*$ 
9:   end for
10: end if
11: return  $(A^*, B^*, \Delta\Phi^*)$ 
```

KQNodes — Algoritmo C4

Algoritmo 3 KQNodes C4 — Refinamiento greedy submodular con MinHeap

Entrada: Subsistema preparado (NCubes, D , N , estado inicial s_0), número de partes k

Salida: k -partición $\Pi^* = \{P_1, \dots, P_k\}$, pérdida Φ^*

```

1: Preparar tensores
2:  $\text{data\_nd} \leftarrow \text{np.stack}([c.\text{data} \ \forall c \in \text{NCubes}])$   $\triangleright$  forma  $(N, 2^D)$ 
3:  $\text{pivot\_idx} \leftarrow (s_0[\text{dim}_d] \ \forall d \in \text{dims\_ncubos})$ 
4:  $\text{pivot\_vals} \leftarrow \text{data\_nd}[:, \text{pivot\_idx}[:, -1]]$   $\triangleright p(\text{nodo}_i = \text{ON} \mid s_0)$ 
5:  $V \leftarrow \{0, 1, \dots, \min(N, D) - 1\}$ 
6: if  $k = 2$  then
7:   return QNodes exacto (regresión DB-03.1)
8: end if
9: Paso 0 — bipartir  $V$  completo
10:  $(A_0, B_0, \varphi_0) \leftarrow \text{QNodesBloque}(V, N, D, \text{data\_nd}, \text{pivot})$ 
11:  $H \leftarrow [(\varphi_0, -|V|, 0, V, A_0, B_0)]$ 
12:  $\Pi \leftarrow [V], \text{id} \leftarrow 0$ 
13: for  $t = 1$  to  $k - 1$  do
14:    $(\varphi^*, \_, \_, P^*, A^*, B^*) \leftarrow \text{HeapPop}(H)$ 
15:    $\Pi \leftarrow (\Pi \setminus \{P^*\}) \cup \{A^*\} \cup \{B^*\}$ 
16:   for  $Q \in \{A^*, B^*\}$  con  $|Q| \geq 2$  do
17:      $(A_Q, B_Q, \varphi_Q) \leftarrow \text{QNodesBloque}(Q, N, D, \text{data\_nd}, \text{pivot})$ 
18:      $\text{id} += 1$ 
19:      $\text{HeapPush}(H, (\varphi_Q, -|Q|, \text{id}, Q, A_Q, B_Q))$ 
20:   end for
21: end for
22: EMD-Effect real (una sola vez)
23:  $\Phi^* \leftarrow \text{CalcPhiTotal}(\Pi, \text{subsistema})$ 
24: return  $(\Pi, \Phi^*)$ 

```

Subrutina QNodesBloque(P) — oracle restringido por bloque:

Algoritmo 4 QNodesBloque — bipartición de un bloque con oracle local

Entrada: Bloque $P \subseteq V$, tensores del subsistema, pivote

Salida: $(A^*, B^*, \varphi_{\text{local}})$

```
1:  $m \leftarrow |P|$ ;  $\text{indices\_gl} \leftarrow \text{sorted}(P)$ 
2: Construir caché local  $\mathcal{C} \leftarrow \{\}$  ▷ local a esta llamada
3: function  $f_{\text{LOCAL}}(\text{máscara\_local } m_\ell)$ 
4:   if  $m_\ell = 0$  or  $m_\ell = 2^m - 1$  then return 0
5:   end if
6:   if  $m_\ell \in \mathcal{C}$  then return  $\mathcal{C}[m_\ell].\varphi$ 
7:   end if
8:   Construir slices sobre data_nd: libre si bit local  $\in$  máscara, fijo al pivote si no
9:    $\bar{\mu}_A \leftarrow \text{data\_nd}[\text{slc\_A}].\text{mean}()$ ;  $\bar{\mu}_B \leftarrow \text{data\_nd}[\text{slc\_B}].\text{mean}()$ 
10:   $\mathcal{C}[m_\ell] \leftarrow (\bar{\mu}_A, \bar{\mu}_B)$ ;  $\mathcal{C}[\overline{m_\ell}] \leftarrow (\bar{\mu}_B, \bar{\mu}_A)$  ▷ complemento gratis
11:  return  $\sum_i \min(|\bar{\mu}_B[i] - v_i|, |\bar{\mu}_A[i] - v_i|)$ 
12: end function
13:  $(\varphi^*, m^*) \leftarrow \text{Queyranne}(m, f_{\text{local}})$  ▷ MAO,  $\mathcal{O}(m^3)$  evaluaciones
14:  $A^* \leftarrow \{\text{indices\_gl}[r] : \text{bit}_r(m^*) = 1\}$ ;  $B^* \leftarrow P \setminus A^*$ 
15: return  $(A^*, B^*, \varphi^*)$ 
```

4.4. Estrategia de búsqueda

KGeoMIP: candidatos guiados por afinidad causal

Para bloques grandes ($2^{|P|-1} - 1 > 4096$), `_mejor_corte_guiado_por_S` genera hasta $N_{\text{máx}} = 20$ candidatos de bipartición ordenados por menor afinidad cruzada entre A y B bajo S :

$$\text{afinidad}(A, B) = \frac{1}{|A| \cdot |B|} \sum_{i \in A, j \in B} S_{ij} \quad (11)$$

Los candidatos se seleccionan mediante `_candidatos_por_afinidad` del módulo `kgeomip_cortes.py`, que combina tres fuentes:

1. **Enumerados** (bloques $|P| \leq 5$): todas las biparticiones posibles.
2. **Constructivos** (bloques medianos): seeds de un elemento, expandidos greedily por menor S_{ij} hasta cubrir la mitad.
3. **Por afinidad** (bloques grandes): cortes que separan clusters de baja similitud entre sí.

De los candidatos generados, solo los primeros $N_{\text{máx}} = 20$ se evalúan con $\Delta\Phi$ real. Esto transforma una búsqueda $\mathcal{O}(2^{|P|})$ en $\mathcal{O}(|P|^2 + 20)$.

KQNodes: oracle MAO de Queyranne

Queyranne (1998) garantiza encontrar la bipartición de mínimo de una función submodular simétrica f en exactamente $\mathcal{O}(m^3)$ evaluaciones de f , donde $m = |P|$. El algoritmo

construye un **Ordenamiento de Máxima Adyacencia (MAO)**: en cada etapa agrega al ordenamiento el elemento con mayor suma de f con los ya incluidos. La bipartición de costo mínimo corresponde al último elemento del MAO junto con el penúltimo como su «testigo».

La clave del funcionamiento correcto en KQNodes es el **caché local por bloque**: cada llamada a `QNodesBloque` crea su propio diccionario de medias $\mathcal{C}$. Dos bloques distintos pueden tener la misma máscara numérica local con semántica diferente; un caché global produciría resultados incorrectos.

Evaluación de la pérdida $\Delta\Phi$ (KGeoMIP)

La descomposición aditiva de EMD-Effect permite calcular el incremento exacto sin recomputar Φ completo:

$$\Delta\Phi(A, B) = \text{costo}(A) + \text{costo}(B) - \text{costo}(A \cup B) \quad (12)$$

donde $\text{costo}(Q) = \sum_{d \in Q} |p_d^{\text{orig}} - p_d^{(Q)}|$ y $p_d^{(Q)}$ es la marginal del nodo d condicionada solo a las dimensiones de Q . Cada $\text{costo}(Q)$ se cachea en `_costo_cache[frozenset(Q)]`: un bloque P aparece en el heap como entero (P) y luego como sus hijos (A, B), de modo que las tres evaluaciones de la Ecuación 12 son hits de caché en la segunda y tercera llamada.

4.5. Evaluación final de la k-partición

Una vez identificada la k-partición $\Pi^* = \{P_1, \dots, P_k\}$, se calcula la pérdida exacta mediante **CalcPhiTotal**:

1. Calcular $\mathbf{p}^{\text{orig}} = \text{distribucion_marginal}()$ del subsistema completo (sin partición).
2. Para cada nodo d en la parte P_m :
 - a) Obtener el NCube de d .
 - b) Marginalizar sobre todas las dimensiones **fuera** de P_m (promedio uniforme sobre las dimensiones del mecanismo que no pertenecen a P_m).
 - c) Seleccionar el valor en el estado pivote s_0 : $p_d^{(\Pi)} \leftarrow \text{NCube}_d[\text{pivot}_{P_m}]$.
3. $\Phi^* \leftarrow \sum_{d=1}^N |p_d^{\text{orig}} - p_d^{(\Pi)}|$

Esta es la EMD-Effect bajo independencia de partes (norma L_1 sobre los vectores de distribución marginal), y se evalúa **una sola vez** para la partición ganadora. El costo es $\mathcal{O}(N \cdot D)$, despreciable frente al refinamiento.

4.6. Optimizaciones implementadas

Cuadro 5: Optimizaciones implementadas en KGeoMIP y KQNodes con su impacto.

Optimización	Descripción	Estrategia	Impacto
Caché de subsistema	Se reutilizan S , <code>flat_nd</code> , sol $k = 2$ y cachés entre llamadas con el mismo (condición, alcance, mecanismo).	KGeoMIP	Evita recalcular S al barrer $k \in \{2, 3, 4, 5\}$
Caché de marginales <code>_marg_cache</code>	Indexado por máscara de bits entera; guarda <code>ndarray(D)</code> por evaluación.	KGeoMIP	$\mathcal{O}(1)$ para máscaras repetidas
Caché de costos <code>_costo_cache</code>	Indexado por <code>frozenset</code> ; guarda el costo real de cada parte.	KGeoMIP	$\Delta\Phi$ en $\mathcal{O}(1)$ con tres lookups
Vista n -dimensional <code>_flat_nd</code>	<code>reshape</code> sin copia de <code>_flat_T</code> ; permite marginalizar por indexación de ejes en lugar de máscaras booleanas sobre 2^D estados.	KGeoMIP	Marginalización en $\mathcal{O}(2^{ P })$ en vez de $\mathcal{O}(2^D)$
S por bloques (OPT-K3)	Multiplicación matricial <code>tabla.T @ difiere</code> en bloques de $\min(2^D, 2^{16})$ filas; usa BLAS internamente.	KGeoMIP	$\sim 10\times$ vs. bucle Python
Caché local por bloque <code>_means_cache</code>	Diccionario nuevo por llamada a <code>_oracle_restringido</code> ; almacena medias $(\bar{\mu}_A, \bar{\mu}_B)$ y el complemento gratis.	KQNodes	Corte la mitad de evaluaciones del oracle
Complemento gratis del oracle	Al calcular $f(m_\ell)$, se guarda también $f(\overline{m}_\ell)$ sin costo adicional (solo se invierten los roles A/B).	KQNodes	Reduce evaluaciones reales a $\sim$ mitad
EMD al final (D4-04 / D3-04)	La EMD-Effect real se evalúa solo sobre la k-partición ganadora, no en cada paso del refinamiento.	Ambas	Reduce llamadas a EMD de $\mathcal{O}(kD^2)$ a $\mathcal{O}(1)$
MinHeap <code>heapq</code>	Mantiene el mínimo en $\mathcal{O}(\log k)$; el heap tiene a lo sumo $k - 1$ elementos en estado estacionario.	Ambas	$\mathcal{O}(k \log k)$ vs. $\mathcal{O}(k^2)$ por reordenamiento

5. Análisis de Complejidad

En esta sección se usan las siguientes variables: n = número de nodos del sistema completo, D = dimensión del subsistema (número de variables presentes/futuras tras substracción), N = número de NCubes (nodos en el alcance del subsistema, $N \leq n$), k = número de partes de la k -partición, m = cardinalidad de un bloque en el refinamiento ($m \leq D$), $S(n, k)$ = número de Stirling de segundo tipo.

5.1. Complejidad temporal

KQNodes — por componente

Cuadro 6: Complejidad temporal de cada componente de KQNodes.

Componente	Complejidad	Descripción
Carga de TPM	$\mathcal{O}(2^n \cdot n)$	Lectura de CSV con <code>np.genfromtxt</code>
<code>sia_preparar_subsistema</code>	$\mathcal{O}(n \cdot 2^n)$	Condicionamiento y substracción del subsistema
<code>_oracle_restringido</code> (bloque m)	$\mathcal{O}(m^2)$ llamadas $\times$ $\mathcal{O}(N \cdot 2^m/2)$	Oracle lazy con caché bidireccional; el complemento gratis reduce evaluaciones reales a la mitad
<code>qnodes(m, f, mask)</code>	$\mathcal{O}(m^3)$ llamadas al oracle	MAO de Queyranne; m^3 en número de evaluaciones de f
<code>_qnodes_sobre_bloque</code> (bloque m)	$\mathcal{O}(m^3 \cdot N \cdot 2^m/2)$	Producto oracle $\times$ MAO
<code>_refinar_c4</code> (k iteraciones)	$\mathcal{O}(k \cdot D^3 \cdot N \cdot 2^D/2)$	$\leq 2k-1$ bloques; bloque máximo $m = D$
<code>_calcular_phi_total</code>	$\mathcal{O}(N \cdot D)$	Una sola evaluación EMD-Effect al final
Total KQNodes $k \geq 3$	$\mathcal{O}(k \cdot D^3 \cdot N)$	Asintótico dominante (absorbe el 2^D del oracle con caché)
Búsqueda exhaustiva	$\mathcal{O}(S(n, k) \cdot N)$	$S(n, k)$ crece superexponencialmente

KGeoMIP — por componente

Cuadro 7: Complejidad temporal de cada componente de KGeoMIP.

Componente	Complejidad	Descripción
GeoMIP (anclaje $k = 2$)	$\mathcal{O}(D \cdot 2^D)$	Construcción de tabla BFS sobre hipercubo; ocurre una sola vez
Construcción de S (D4-02)	$\mathcal{O}(D^2 \cdot 2^D)$	Multiplicación matricial por bloques <code>tabla.T @ difiere</code> ; calculada una vez por subsistema
<code>_marginales_mascara</code> (por máscara)	$\mathcal{O}(2^{ P })$ sin caché; $\mathcal{O}(1)$ con caché	Indexación sobre vista n -dimensional <code>_flat_nd</code>
<code>_delta_phi_corte(A, B)</code>	$\mathcal{O}(1)$ con cachés llenos; $\mathcal{O}(2^D)$ primera vez	Tres lookups de <code>_costo_cache</code>
MejorCorte (bloque m , modo auto)	$\mathcal{O}(2^{m-1})$ si $m \leq 13$; $\mathcal{O}(m^2 + 20)$ si $m > 13$	Umbral $U = 4096$: exhaustivo para $2^{m-1} - 1 \leq 4096$
<code>_refinar_e4</code> (k iteraciones)	$\leq 2(k-1)$ llamadas a MejorCorte	Cada llamada: $\mathcal{O}(2^{m-1} \cdot 2^D)$ o $\mathcal{O}(m^2 \cdot 2^D)$
<code>_calcular_phi_total</code>	$\mathcal{O}(N \cdot D)$	Una sola evaluación EMD-Effect al final
Total KGeoMIP $k \geq 3$	$\mathcal{O}(k \cdot D^2 \cdot 2^D)$	Dominado por la construcción de S y el refinamiento
Búsqueda exhaustiva	$\mathcal{O}(S(n, k) \cdot N)$	Referencia de comparación

Operaciones dominantes e identificación de cuellos de botella

Para KQNodes, la operación dominante es **la evaluación del oracle** en cada bloque: cada llamada a `_qnodes_sobre_bloque` requiere $\mathcal{O}(m^3)$ evaluaciones de f_{local} , y cada evaluación accede a un tensor de forma $(N, 2^m)$. Para $N = 20$ y $m = 20$, el tensor tiene ~ 20 millones de entradas; con caché, el número de evaluaciones distintas se reduce a $\mathcal{O}(m^2/2)$ (complemento gratis).

Para KGeoMIP, el cuello de botella es **la construcción de S** : costo $\mathcal{O}(D^2 \cdot 2^D)$, ejecutado una sola vez por subsistema. Para $D = 25$: $625 \times 2^{25} \approx 2 \times 10^{10}$ operaciones; mitigado por la implementación BLAS en bloques de 2^{16} estados.

5.2. Complejidad espacial

Cuadro 8: Uso de memoria de las principales estructuras de datos.

Estructura	Tamaño	Notas
TPM cargada	$\mathcal{O}(2^n \cdot n)$	Permanente; ~ 40 MB para $n = 20$
Subsistema (System + NCu- bes)	$\mathcal{O}(N \cdot 2^D)$	N NCubes de forma $(2, \dots, 2)$ con D ejes
Tabla T (_flat_T)	$\mathcal{O}(2^D \cdot D)$	GeoMIP; ~ 800 MB para $D = 25$ (float32)
Vista _flat_nd	$\mathcal{O}(1)$ adicional	Es un reshape sin copia de _flat_T
Matriz S	$\mathcal{O}(D^2)$	D^2 entradas float64; < 5 KB para $D = 25$
_marg_cache	$\mathcal{O}(2^D \cdot D)$ en el peor caso	A lo sumo 2^D máscaras distintas; cada una almacena un vector de D floats
_costo_cache	$\mathcal{O}(2^D)$ en el peor caso	A lo sumo 2^D frozensets distintos; valor es un float
MinHeap E4 / C4	$\mathcal{O}(k)$	El heap tiene a lo sumo $k-1$ elementos en estado estacionario
_means_cache (oracle KQNodes)	$\mathcal{O}(2^m)$ por bloque	Local a cada llamada; se descarta al terminar el bloque
Total KGeoMIP	$\mathcal{O}(2^D \cdot D)$	Dominado por _flat_T
Total KQNodes	$\mathcal{O}(N \cdot 2^D)$	Dominado por data_nd

5.3. Análisis de casos

Mejor caso

El mejor caso para ambas estrategias es $k = 2$:

- **KGeoMIP**: retorna directamente la solución de GeoMIP sin construir S ni ejecutar el refinamiento. Costo total: $\mathcal{O}(D \cdot 2^D)$.
- **KQNodes**: ejecuta Queyranne una sola vez sobre V completo. Costo total: $\mathcal{O}(D^3 \cdot N)$.

Ambos son **exactos** para $k = 2$ (verificado con $|\Delta\varphi| < 10^{-9}$).

El mejor caso dentro de $k \geq 3$ ocurre cuando **todos los bloques del refinamiento son de tamaño 1 o 2**: el heap se vacía prematuramente porque no quedan partes partibles, lo que termina el refinamiento antes de llegar a k partes. Esto sucede cuando $D < k$ (más partes pedidas que dimensiones disponibles).

Peor caso

El peor caso para **KQNodes** es $k = D$ (atomización total, cada dimensión en su propia parte) con $m = D$ en el primer bloque: se ejecutan $\mathcal{O}(D^3)$ evaluaciones del oracle sobre el bloque completo, cada una sobre tensores de tamaño $\mathcal{O}(N \cdot 2^D)$. Costo total: $\mathcal{O}(D \cdot D^3 \cdot N) = \mathcal{O}(D^4 \cdot N)$.

Ejemplo ($n = 5, D = 5, N = 5, k = D = 5$): El algoritmo hace $k - 1 = 4$ iteraciones. El bloque inicial tiene $m = 5$ y Queyranne realiza $\mathcal{O}(5^3) = 125$ evaluaciones del oracle, cada una sobre un tensor $(5 \times 2^5) = 160$ valores. Los bloques hijos son $m = 3$ (27 eval.), $m = 2$ (8 eval.) y $m = 1$ (trivial). Total: $125 + 27 + 8 \approx 160$ evaluaciones, frente a $S(5, 5) = 1$ partición exhaustiva — en este caso el peor caso es de pocos nodos y la ventaja no se aprecia. Para $n = 8, k = D = 8$: Queyranne en el primer bloque cuesta $8^3 = 512$ evaluaciones y el total escala a $\mathcal{O}(8^4 \times 8) \approx 26\,000$ operaciones, mientras la exhaustiva requeriría $S(8, 8) \times 8 = 1 \times 8 = 8$ (trivial para $k = n$). El peor caso práctico ocurre en $k < n$ con k cercano a D : para $n = 10, k = 8, D = 10$: $\mathcal{O}(8 \times 10^3 \times 10) = 80\,000$ operaciones.

El peor caso para **KGeoMIP** es cuando todos los bloques del refinamiento superan el umbral $U = 4096$ (modo guiado_S), lo que requiere generar candidatos constructivos $\mathcal{O}(m^2)$ para cada bloque. Para $k = 5$ sobre $D = 25$: ≤ 8 bloques $\times \mathcal{O}(25^2) = 625$ candidatos $\times \mathcal{O}(2^{25})$ por $\Delta\Phi$: dominante es la construcción de S en $\mathcal{O}(D^2 \cdot 2^D)$.

Ejemplo ($n = 20, D = 20, k = 3$):

- **Construcción de S** : $D^2 \times 2^D = 400 \times 2^{20} \approx 4 \times 10^8$ operaciones (ejecutadas una sola vez con BLAS).
- **Refinamiento E4**: ≤ 4 llamadas a **MejorCorte**. Cada bloque tiene $m \approx 10$ elementos; $2^{10-1} - 1 = 511 \leq 4096$, así que aplica el modo exhaustivo (no guiado_S). Costo por bloque: $511 \times \mathcal{O}(2^{20}) \approx 5 \times 10^8$ operaciones.
- **Comparación**: la construcción de S (4×10^8) y cada bloque exhaustivo (5×10^8) tienen el mismo orden — ambos dominan. La búsqueda exhaustiva requeriría $S(20, 3) \approx 580 \times 10^6$ evaluaciones EMD completas: inviable en tiempo real.

Las **características que conducen al peor caso** son:

- k grande (cercano a D): más iteraciones del heap y bloques más pequeños que no aceleran el oracle.
- D grande con bloque inicial de tamaño D : el primer bloque del heap es el más costoso (tamaño máximo).
- Pocos empates en $\Delta\Phi / \varphi_{\text{local}}$: el heap no puede reutilizar evaluaciones previas.

5.4. Comparación con alternativas

Frente a la búsqueda exhaustiva

Cuadro 9: Comparación de complejidad y evaluaciones concretas para distintos (n, k) . Tiempos estimados en hardware de escritorio (Python, numpy).

n	k	Exhaustiva (EMDs)	KQNodes C4 (ops)	KGeoMIP E4 (ops)
8	3	$966 \times 8 = 7\,728$	$\leq 9 \times 8^3 \times 8 \approx 4\,608$	$\leq 8 \times 64 \times 2^8 \approx 131\,072$
8	3	~ 2 s	~ 0.01 s	~ 0.05 s
10	4	$145\,750 \times 10$	$\leq 7 \times 10^3 \times 10 = 700\,000$	$\leq 6 \times 100 \times 2^{10} \approx 614\,400$
10	4	<i>inviabile (>1 min)</i>	~ 0.1 s	~ 0.5 s
20	3	$\sim 580 \times 10^6$	$\leq 9 \times 20^3 \times 20 = 720\,000$	$\leq 4 \times 400 \times 2^{20} \approx 1.7 \times 10^9$
20	3	<i>semanas</i>	~ 1 s	~ 30 s

Frente a las estrategias originales de bipartición

Cuadro 10: Comparación entre las extensiones k-particiones y sus anclas de bipartición.

Algoritmo	Complejidad temporal	Complejidad espacial
GeoMIP (k=2)	$\mathcal{O}(D \cdot 2^D)$	$\mathcal{O}(2^D \cdot D)$
KGeoMIP E4 (k>2)	$\mathcal{O}(k \cdot D^2 \cdot 2^D)$	$\mathcal{O}(2^D \cdot D)$
Overhead KGeoMIP	factor $k \cdot D$ sobre GeoMIP	igual
QNodes (k=2)	$\mathcal{O}(D^3 \cdot N)$	$\mathcal{O}(N \cdot 2^D)$
KQNodes C4 (k>2)	$\mathcal{O}(k \cdot D^3 \cdot N)$	$\mathcal{O}(N \cdot 2^D)$
Overhead KQNodes	factor k sobre QNodes	igual

El overhead de extender a k-particiones es **lineal en k** para KQNodes y **lineal en $k \cdot D$** para KGeoMIP, frente al crecimiento superexponencial $S(n, k) \sim k^n/k!$ de la búsqueda exhaustiva. Para $k = 5$ y $D = 20$, esto representa un factor de reducción superior a 10^8 evaluaciones.

Comparación entre KGeoMIP y KQNodes

Ninguna de las dos estrategias domina a la otra en todas las dimensiones:

- **KQNodes es preferible** cuando D es moderado ($D \leq 15$) y N es pequeño: la complejidad $\mathcal{O}(k \cdot D^3 \cdot N)$ crece solo polinomialmente con D , sin el factor exponencial 2^D .
- **KGeoMIP es preferible** cuando D es grande ($D > 15$) pero el umbral de enumeración permite modos exactos: la tabla T ya construida por GeoMIP se reutiliza sin coste adicional, y S se construye una sola vez para todos los k del barrido.

- En **memoria**, ambas tienen el mismo orden $\mathcal{O}(N \cdot 2^D)$ o $\mathcal{O}(2^D \cdot D)$ — el cuello de botella para $D > 22$ en ambos casos.

6. Detalles de Implementación

6.1. Métodos Principales

A continuación se documentan los métodos públicos y funciones auxiliares más relevantes del proyecto, con sus firmas exactas, parámetros y valores de retorno.

KGeoMIP.aplicar_estrategia

```

1 def aplicar_estrategia(
2     self,
3     condicion: str,
4     alcance: str,
5     mecanismo: str,
6     tpm: np.ndarray,
7     k: int = 2,
8     variante: str = "E4",
9     estrategia_corte: str = "auto",
10 ) -> Solution:

```

Descripción: Halla la k-partición de mínima información integrada usando la estrategia KGeoMIP con anclaje en GeoMIP. Para $k = 1$ retorna $\varphi = 0$ directamente; para $k = 2$ retorna la solución de GeoMIP sin cálculo adicional.

Parámetro	Tipo	Descripción
condicion	str	Máscara binaria; bit=1 → no-do candidato
alcance	str	Máscara binaria del alcance del subsistema
mecanismo	str	Máscara binaria del mecanismo del subsistema
tpm	np.ndarray	TPM precargada de forma $(2^n, n)$
k	int	Número de partes ($1 \leq k \leq 5$)
variante	str	"E4" (divisiva guiada) o "A" (aglomerativa)
estrategia_corte	str	"auto" / "exhaustivo" / "guiado_S"
Retorno	Solution	perdida, particion, tiempo_total

Comportamiento especial:

- Si la clave (`condicion`, `alcance`, `mecanismo`) coincide con la llamada anterior, reutiliza la solución GeoMIP anclada, la matriz S y todos los cachés, evitando recomputación al barrer $k \in \{2, 3, 4, 5\}$ sobre el mismo subsistema.
- `estrategia_corte = "auto"` usa modo exhaustivo si $2^{m-1} - 1 \leq 4096$ (bloques de hasta 13 nodos), y guiado_S en bloques más grandes.

`KQNodes.aplicar_estrategia`

```
1 def aplicar_estrategia(  
2     self,  
3     estado_inicial: str,  
4     condicion: str,  
5     alcance: str,  
6     mecanismo: str,  
7     k: int = 2,  
8     criterio: str = "C4",  
9 ) -> Solution:
```

Descripción: Halla la k -partición de mínima información integrada mediante refinamiento iterativo greedy con el oracle de Queyranne. Para $k = 2$ replica exactamente `QNodes.winner()`.

Parámetro	Tipo	Descripción
<code>estado_inicial</code>	<code>str</code>	Estado inicial del sistema (string de bits)
<code>condicion</code>	<code>str</code>	Máscara binaria de nodos candidatos
<code>alcance</code>	<code>str</code>	Máscara binaria del alcance
<code>mecanismo</code>	<code>str</code>	Máscara binaria del mecanismo
<code>k</code>	<code>int</code>	Número de partes ($2 \leq k \leq 5$)
<code>criterio</code>	<code>str</code>	"C4" (mín φ_{local}) o "C1" (máx tamaño)
Retorno	<code>Solution</code>	<code>perdida</code> , <code>particion</code> , <code>tiempo_total</code>

`SIA.sia_preparar_subsistema`

```
1 # Variante GeoMIP (sin estado_inicial; lo gestiona Manager):  
2 def sia_preparar_subsistema(  
3     self, condicion: str, alcance: str, mecanismo: str  
4 ) -> None:  
5
```

```

6 # Variante QNodes:
7 def sia_preparar_subsistema(
8     self, estado_inicial: str, condicion: str,
9     alcance: str, mecanismo: str
10 ) -> None:

```

Descripción: Método de la clase base SIA (patrón *Template Method*). Construye el objeto `System` condicionado y sustraído desde las máscaras binarias. El subsistema resultante se almacena en `sia_subsistema`; las distribuciones marginales en `sia_dists_marginales`. No tiene valor de retorno.

`NCube.marginalizar`

```

1 def marginalizar(self, ejes: NDArray[np.int8]) -> "NCube":

```

Descripción: Colapsa el hipercubo sobre los ejes indicados promediando las dimensiones correspondientes. Retorna un nuevo `NCube` con `dims` reducidos. Es la operación primitiva de la preparación del subsistema y de `_calcular_phi_total`.

- **Parámetro:** `ejes` — array de índices globales de dimensión a eliminar.
- **Retorno:** nuevo `NCube` con `data` de forma reducida y `dims` actualizado.

`Manager.cargar_red`

```

1 def cargar_red(self) -> np.ndarray:

```

Descripción: Lee el CSV de TPM desde `data/samples/N{n}{muestra}.csv` usando `np.genfromtxt` con delimitador `","`.

- **Retorno:** `ndarray` de forma $(2^n, n)$ con la matriz de probabilidad de transición.
- **Excepción:** `FileNotFoundError` si el archivo CSV no existe — propagada sin atrapar para fallo rápido visible. La variable de entorno `GEOMIP_SAMPLES_DIR` debe estar configurada antes de instanciar `Manager` (lo hace `main_kgeomip.py` automáticamente).

`_calcular_phi_total`

```

1 def _calcular_phi_total(
2     particion: list[frozenset[int]],
3     sistema: System,
4 ) -> float:

```

Descripción: Calcula Φ^* con una única llamada a `emd_efecto`:

$$\Phi^* = \text{EMD} \left(p(s_{t+1}), \bigotimes_{P_i} p_{P_i} \right)$$

Para cada nodo d en la parte P_i , reconstruye su distribución marginal via `NCube.marginalizar` eliminando las dimensiones fuera de P_i . Nodos no cubiertos (cuando $N > D_{\text{part}}$) se tratan como singletons.

Retorno: Φ^* como `float`. Esta función es la **autoridad final** de Φ — el $\Delta\Phi$ incremental puede diferir hasta 10^{-6} por acumulación de redondeo flotante.

`_oracle_restringido`

```

1 def _oracle_restringido(
2     bloque: frozenset[int],
3     N: int,
4     D_global: int,
5     data_nd: np.ndarray,
6     pivot_idx: tuple[int, ...],
7     pivot_vals: np.ndarray,
8 ) -> tuple[Callable[[int], float], int]:

```

Descripción: Construye el oracle local para el algoritmo de Queyranne restringido al bloque `bloque`. Retorna la función $f_{\text{local}}(\text{mask})$ y la máscara completa del bloque.

El caché de medias `_means_cache` es **local a cada llamada** y se reinicia por bloque. Esto es correcto porque dos bloques distintos P_i y P_j pueden tener la misma máscara numérica local (e.g., `0b01`) pero con semántica global diferente (decisión D3-02).

6.2. Dependencias Externas

Las dependencias se declaran en `code/pyproject.toml` bajo `[project.dependencies]`. El entorno requiere **Python ≥ 3.12** .

Biblioteca	Versión mín.	Uso en el proyecto
numpy	2.0.2	Álgebra lineal y arrays multidimensionales (NCube, TPM, matriz S). Indexación por máscara de bits y productos matriciales vectorizados.
scipy	1.17.0	Clustering jerárquico (<code>linkage/fcluster</code>) en la variante aglomerativa "A" de KGeoMIP. Importado localmente para no bloquear la carga del módulo.
pandas	2.3.3	Lectura del Excel de pruebas via <code>pd.read_excel</code> y escritura de resultados en CSV con <code>DataFrame.to_csv</code> .
openpyxl	3.1.5	Motor <code>.xlsx</code> requerido por pandas. No se usa directamente en el código del proyecto.
pyinstrument	5.1.2	Profiling estadístico; se activa con <code>profiler_habilitado = True</code> y genera reportes de tiempo por función.
pyphi	1.2.0	Cálculo de la EMD-Effect (<code>emd_efecto</code>) desde <code>pyphi.distance</code> . Proporciona la norma L_1 entre distribuciones causales.
colorama	0.4.6	Coloreado de consola en <code>SafeLogger</code> : DEBUG=gris, INFO=azul, WARNING=amarillo, ERROR=rojo, CRITICAL=magenta.
pyttsx3	2.99	Síntesis de voz opcional en <code>Solution</code> . Por defecto desactivada en KGeoMIP y KQNodes.

Cuadro 11: Dependencias externas declaradas en `code/pyproject.toml`.

6.3. Aspectos de Ingeniería de Software

Logging estructurado — SafeLogger

El módulo `src/middlewares/slogger.py` define la clase `SafeLogger`, un wrapper sobre `logging.Logger` con las siguientes propiedades:

- **Codificación segura:** antes de emitir cada mensaje serializa cualquier objeto a `str` con codificación UTF-8 y sustitución de errores (`errors="replace"`), evitando excepciones por caracteres especiales en strings de partición.
- **Tres destinos** por instancia: (a) archivo detallado organizado por fecha/hora bajo `logs/` (nivel `DEBUG`), (b) archivo de última ejecución `last_{name}.log` en la raíz de `logs/`, y (c) consola con colores via `colorama`.
- **Niveles expuestos:** `debug`, `info`, `warn`, `error`, `critic`, `fatal`. El nivel mínimo de emisión está configurado a `ERROR` por defecto.
- **Singleton por nombre:** `logging.getLogger(name)` garantiza que múltiples instancias de KGeoMIP en el mismo proceso compartan el mismo logger sin duplicar handlers.

Ejemplo de uso en KGeoMIP:

```
1 self.logger = SafeLogger("KGeoMIP")
2 self.logger.warn("heap agotado en _refinar_e4, k =", k)
3 self.logger.debug("S construida en", round(t1 - t0, 3), "s")
```

Validación de inputs

Los parámetros se validan en las capas externas (scripts `exec_k*.py`) antes de llegar al núcleo de cálculo:

- **Máscaras binarias:** `condicion`, `alcance` y `mecanismo` deben ser strings de longitud n con solo '0'/'1'. La función `_letras_a_binario` convierte la notación del Excel (e.g., "ABC") a máscara binaria (e.g., "11100000") antes de cualquier llamada al núcleo.
- **Rango de k:** KGeoMIP acepta $1 \leq k \leq 5$; KQNodes acepta $2 \leq k \leq 5$. Los scripts de entrada restringen los valores configurables; no hay excepción explícita dentro del núcleo.
- **Límite D_{part} :** internamente se usa $D_{\text{part}} = \min(N, D)$ como número efectivo de dimensiones, evitando que $k > D$ genere bloques vacíos o índices fuera de rango.

Manejo de errores

- `FileNotFoundError`: si `N{n}{muestra}.csv` no existe en el directorio de muestras, `np.genfromtxt` lo propaga sin atrapar — fallo rápido visible al operador.

- **Heap agotado en KGeoMIP:** si el MinHeap se vacía antes de completar $k - 1$ iteraciones, `_refinar_e4` emite `self.logger.warn` y retorna la partición parcial disponible.
- **Bloque de tamaño 1:** `_qnodes_sobre_bloque` retorna `(bloque, frozenset(), 0.0)` sin invocar `Queyranne`, evitando el caso degenerado de un oracle con un único elemento.
- **Subsistema vacío:** si `sistema.indices_ncubos` o `sistema.dims_ncubos` está vacío tras el condicionamiento, `KQNodes._resolver` retorna `Solution` con $\varphi = 0$ y `particion=""` sin entrar al refinamiento.

Profiling y debugging

El singleton `aplicacion` centraliza la configuración global del entorno:

```
1 aplicacion.profiler_habilitado = True # activa pyinstrument
2 aplicacion.pagina_sample_network = "A" # selecciona N{n}A.csv
```

Cuando el profiling está activo, los scripts `exec_k*.py` envuelven la llamada `batch` en un contexto `pyinstrument.Profiler` y emiten el reporte de tiempos por función al finalizar. Esto permite identificar cuellos de botella sin instrumentar el código de cálculo manualmente.

La estrategia de debugging recomendada es:

1. Activar `SafeLogger` a nivel `DEBUG` para ver el flujo interno (tamaños de bloque, $\Delta\Phi$ por iteración, cache hits/misses).
2. Usar `pyinstrument` para identificar la función dominante en tiempo.
3. Comparar `_calcular_phi_total` vs $\sum \Delta\Phi$ acumulado: diferencias $> 10^{-6}$ indican un bug en la lógica incremental.

6.4. Tests Implementados

Los tests se organizan en la carpeta `tests/` de cada submódulo.

`test_regresion_k2_igual_qnodes`

Tipo: regresión / integración.

Objetivo: verificar que `KQNodes` ($k=2$) produce exactamente el mismo φ que `QNodes` para todos los casos del Excel de pruebas.

Casos: 13 casos para $n \in \{5, 8, 10\}$, distintas combinaciones de alcance y mecanismo.

Criterio de éxito: $|\Delta\varphi| < 10^{-9}$ en todos los casos.

Resultado: 13/13 verificados (ver `context/handoffs/03.md`).

test_regresion_k2_igual_geomip

Tipo: regresión / integración.

Objetivo: verificar que KGeoMIP(k=2) produce exactamente el mismo φ que GeoMIP (garantía D4-01).

Casos: $n \in \{5, 8\}$ con estados iniciales "10000" y "100000000".

Criterio de éxito: $|\Delta\varphi| < 10^{-9}$.

test_ab_c1_vs_c4_n5

Tipo: A/B / calidad de heurística.

Objetivo: comparar el criterio C4 (mín φ_{local}) contra C1 (máx tamaño de parte) para $k \in \{3, 4, 5\}$ con $n = 5$.

Métrica: gap relativo respecto al óptimo obtenido por búsqueda exhaustiva ($S(5, k)$ pequeño).

Resultado esperado: C4 produce gap $\leq$ C1 en la mayoría de los casos; empíricamente el gap de C4 es 0 para $k = 3$.

test_monotonicidad_creciente

Tipo: propiedad / integración.

Objetivo: verificar que $\varphi(k+1) \geq \varphi(k)$ para todo k con $n = 5$, $k \in \{2, 3, 4, 5\}$.

Criterio de éxito: ningún caso viola la monotonicidad (tolerancia 10^{-9}).

Tests de $\Delta\Phi$ incremental (D4-06)

Tipo: precisión numérica / unitario.

Objetivo: verificar que `_delta_phi_corte(A, B)` es igual a $\Phi(\Pi') - \Phi(\Pi)$ calculado con `_calcular_phi_total`.

Criterio de éxito: diferencia $< 10^{-6}$ para todos los cortes del refinamiento en $n \in \{5, 8\}$.

Estrategia de validación

La validación del proyecto opera en tres niveles complementarios:

1. **Regresión** $k = 2$: el caso base exacto garantiza que la extensión no rompe el comportamiento ya validado de GeoMIP y QNodes.
2. **Exhaustiva para** $n \leq 5$: resultados de KGeoMIP/KQNodes se comparan contra búsqueda exhaustiva sobre los $S(n, k)$ posibles, cuantificando el gap de la heurística.
3. **Propiedades estructurales**: monotonicidad y correctitud del $\Delta\Phi$ incremental se verifican como invariantes del algoritmo, detectando regresiones en modificaciones futuras al núcleo.

7. Resultados Experimentales

7.1. Datasets Utilizados

Los experimentos utilizan los siguientes recursos de datos:

- **Archivo de casos de prueba:** `data/DatosPruebas2026_1.xlsx` con 7 hojas (una por tamaño de red $n \in \{5, 8, 10, 15, 20, 22, 25\}$). Cada hoja contiene 49–50 filas con pares (`Alcance`, `Mecanismo`) codificados en notación de letras (A–Z).
- **Redes bayesianas de prueba:** archivos `data/samples/N{n}{M}.csv`, matrices de probabilidad de transición (TPM) de forma $(2^n, n)$, donde $M \in \{A, B\}$ identifica la muestra de red. Los tamaños de archivo van desde ≈ 0.5 MB ($n = 8$) hasta ≈ 640 MB ($n = 20$).

Se ejecutaron en total **18 configuraciones experimentales**: 12 con KQNodes C4 y 6 con KGeoMIP E4, cubriendo $n \in \{8, 10, 15, 20\}$, $k \in \{3, 4, 5\}$ y muestras A y B. Todas las ejecuciones usan el estado inicial "100...0" (nodo A activo, resto en cero).

7.2. Métricas de Evaluación

Se reportan las siguientes métricas por cada ejecución:

- **Pérdida φ :** valor de la función objetivo EMD-Effect = $\text{EMD}(p(s_{t+1}), \bigotimes_{P_i} p_{P_i})$ calculado por `_calcular_phi_total` sobre la k -partición encontrada. Valores altos indican que la partición rompe fuertemente la causalidad integrada.
- **Tiempo de ejecución t (s):** tiempo de pared desde la llamada a `aplicar_estrategia` hasta el retorno de `Solution`.
- **Número de pruebas:** casos del Excel procesados por configuración (49–50).

7.3. Tablas de Resultados

Tabla 1 — Resumen ejecutivo completo (18 experimentos)

Algoritmo	n , Muestra, k	Casos	$\varphi_{\min}$	φ_{avg}	$\varphi_{\max}$	t_{avg} (s)
<i>KQNodes C4</i>						
KQNodes C4	8, A, 5	49	0.0000	1.7959	4.0000	0.0038
KQNodes C4	10, A, 3	49	0.1914	1.4647	4.2480	0.0055
KQNodes C4	10, A, 4	49	0.1582	1.5185	4.2949	0.0061
KQNodes C4	10, A, 5	49	0.1992	1.5444	4.5254	0.0068
KQNodes C4	15, A, 3	50	0.0393	0.1458	0.3460	0.0278
KQNodes C4	15, A, 4	50	0.0491	0.1760	0.3478	0.0305
KQNodes C4	15, A, 5	50	0.0527	0.2029	0.3800	0.0578
KQNodes C4	15, B, 3	50	0.0025	0.0129	0.0292	0.0264
KQNodes C4	15, B, 4	50	0.0037	0.0156	0.0275	0.0301
KQNodes C4	15, B, 5	50	0.0045	0.0180	0.0297	0.0627
KQNodes C4	20, A, 4	50	0.0820	2.6611	8.7807	1.4985
KQNodes C4	20, A, 5	50	0.0823	2.6467	9.2800	3.1746
<i>KGeoMIP E4</i>						
KGeoMIP E4	10, A, 3	49	0.0283	1.0156	3.3848	0.0252
KGeoMIP E4	15, B, 3	50	0.0004	0.0074	0.0209	0.4556
KGeoMIP E4	15, B, 4	50	0.0008	0.0105	0.0236	0.7986
KGeoMIP E4	15, B, 5	50	0.0013	0.0138	0.0269	0.4633
KGeoMIP E4	20, A, 3	50	0.0007	2.2177	8.2487	6.1883
KGeoMIP E4	20, A, 4	50	0.0170	2.3926	8.8741	3.4927

Cuadro 12: Resumen ejecutivo de las 18 configuraciones experimentales. Estado inicial "100...0", variante E4 (KGeoMIP) y C4 (KQNodes).

Tabla 2 — Efecto de k sobre φ (monotonicidad)

La Tabla 13 muestra cómo φ_{avg} crece con k para configuraciones con tres valores de k disponibles, verificando empíricamente la Proposición 1 (§2.9.1).

Algoritmo	n , Muestra	φ_{avg} ($k = 3$)	φ_{avg} ($k = 4$)	φ_{avg} ($k = 5$)
KQNodes C4	10, A	1.4647	1.5185	1.5444
KQNodes C4	15, A	0.1458	0.1760	0.2029
KQNodes C4	15, B	0.0129	0.0156	0.0180
KGeoMIP E4	15, B	0.0074	0.0105	0.0138

Cuadro 13: φ_{avg} en función de k para configuraciones con $k \in \{3, 4, 5\}$. En todos los casos $\varphi(k+1) \geq \varphi(k)$, confirmando la monotonicidad por construcción.

Tabla 3 — Escalabilidad temporal: t_{avg} vs n

La Tabla 14 muestra el crecimiento del tiempo promedio con el tamaño de la red, fijando $k = 5$ para KQNodes y $k = 3$ para KGeoMIP.

Algoritmo	Muestra	$t_{\text{avg}} n = 8$	$t_{\text{avg}} n = 10$	$t_{\text{avg}} n = 15$	$t_{\text{avg}} n = 20$
KQNodes C4 ($k = 5$)	A	0.0038 s	0.0068 s	0.0578 s	3.1746 s
KQNodes C4 ($k = 5$)	B	—	—	0.0627 s	—
KGeoMIP E4 ($k = 3$)	A	—	0.0252 s	—	6.1883 s
KGeoMIP E4 ($k = 3$)	B	—	—	0.4556 s	—

Cuadro 14: Tiempo promedio (s) por configuración en función de n . KQNodes muestra A $k = 5$: factor $\approx 467\times$ entre $n = 10$ y $n = 20$. KGeoMIP: construcción de S domina el costo para $n = 15$.

Tabla 4 — Comparación KGeoMIP vs KQNodes ($n = 15$, muestra B)

La Tabla 15 compara ambas estrategias sobre la misma red y los mismos casos de prueba, para $k \in \{3, 4, 5\}$.

k	φ_{avg} E4	φ_{avg} C4	Diferencia (%)	t_{avg} E4	t_{avg} C4
3	0.0074	0.0129	−42 %	0.456 s	0.026 s
4	0.0105	0.0156	−33 %	0.799 s	0.030 s
5	0.0138	0.0180	−23 %	0.463 s	0.063 s

Cuadro 15: KGeoMIP E4 vs KQNodes C4 sobre $n = 15$, muestra B. KGeoMIP obtiene menor φ en todos los k a costa de un tiempo $\approx 7\text{--}17\times$ mayor por caso.

Tabla 5 — Efecto de la muestra de red ($n = 15$, KQNodes C4)

La Tabla 16 compara los resultados sobre la misma hoja del Excel ($n = 15$) con dos redes distintas: muestra A y muestra B.

k	φ_{avg} A	φ_{avg} B	Factor A/B	t_{avg} A
3	0.1458	0.0129	11.3 $\times$	0.0278 s
4	0.1760	0.0156	11.3 $\times$	0.0305 s
5	0.2029	0.0180	11.3 $\times$	0.0578 s

Cuadro 16: KQNodes C4 sobre $n = 15$: muestra A vs muestra B. La muestra B produce φ sistemáticamente $11\times$ menor, lo que refleja que la red N15B tiene menor información causal integrada. Los tiempos de ejecución son similares entre muestras.

Casos representativos — KGeoMIP $n = 10, k = 3$

#	Alcance	Mecanismo	φ	t (s)
1	ABCDEFGHJIJ	ABCDEFGHJIJ	2.9746	0.0580
8	ABCDEFGFGHI	ABCDEFGFGHIJ	3.3848	0.0390
17	BCDEFGHIJ	BCDEFGHIJ	1.6055	0.0600
29	ABDEGHJ	ABCDEFGFGHIJ	3.2109	0.0190
41	ACEGI	ACEGI	0.2363	0.0090
49	BDFHJ	BDFHJ	0.1328	0.0070

Cuadro 17: Selección de casos para KGeoMIP ($n = 10, k = 3, E4$). Las pruebas con alcance máximo (10 nodos) producen φ alto; los subsistemas de 5 nodos producen φ bajo y tiempo menor.

Casos representativos — KQNodes $n = 20, k = 5$

#	Alcance	Mecanismo	φ	t (s)
1	ABCDEFGHGIJKLMNOPQRST	ABCDEFGHGIJKLMNOPQRST	9.1250	24.63
2	ABCDEFGHGIJKLMNOPQRST	ABCDEFGHGIJKLMNOPQRS	4.0328	10.17
3	ABCDEFGHGIJKLMNOPQRST	BCDEFGHGIJKLMNOPQRST	6.1262	11.98
4	ABCDEFGHGIJKLMNOPQRST	BCDEFGHGIJKLMNOPQRS	3.2818	5.84

Cuadro 18: Primeros 4 casos para KQNodes ($n = 20, k = 5, C4$). El subsistema completo (20 nodos) es el de mayor costo: $2^{20} = 1\,048\,576$ estados.

7.4. Análisis de Resultados

Monotonidad de φ con k

La Tabla 13 confirma empíricamente la Proposición 1 (§2.9.1): en las cuatro configuraciones con $k \in \{3, 4, 5\}$, φ_{avg} crece estrictamente con k . Para KQNodes $n = 10$ A el incremento de $k = 3$ a $k = 5$ es de apenas +5.4 % ($1.465 \rightarrow 1.544$), mientras que para KQNodes $n = 15$ A el incremento es del 39 % ($0.146 \rightarrow 0.203$). Esto indica que en N10A el sistema ya está muy integrado desde $k = 3$, mientras que en N15A cada partición adicional revela más independencia causal.

El efecto de k sobre el tiempo de ejecución es leve: KQNodes $n = 10$ A pasa de 0.0055 s ($k = 3$) a 0.0068 s ($k = 5$), un incremento del 24 % coherente con el costo de $k-1$ llamadas adicionales a Queyranne.

Escalabilidad temporal con n

La Tabla 14 muestra el crecimiento exponencial del tiempo con n para KQNodes C4 ($k = 5$, muestra A):

- $n = 8 \rightarrow n = 10$: factor $1.8\times$ (0.0038 a 0.0068 s).
- $n = 10 \rightarrow n = 15$: factor $8.5\times$ (0.0068 a 0.0578 s).
- $n = 15 \rightarrow n = 20$: factor $54.9\times$ (0.0578 a 3.1746 s).
- $n = 10 \rightarrow n = 20$ acumulado: factor $\approx 467\times$.

El modelo $\mathcal{O}(k \cdot D^3 \cdot N)$ predice un factor teórico $(20/10)^3 \cdot 2^{10} \approx 8192\times$ entre $n = 10$ y $n = 20$. El factor observado ($467\times$) es menor porque no todos los casos tienen alcance completo: los subsistemas con pocos nodos activos terminan en milisegundos aunque $n = 20$.

Para KGeoMIP E4, la construcción de la matriz S (costo $O(n^2 \cdot 2^n)$, calculada una sola vez por subsistema) domina el tiempo para $n = 15$: 0.456 s promedio vs 0.026 s de KQNodes en la misma configuración (muestra B, $k = 3$).

Comparación entre estrategias: KGeoMIP E4 vs KQNodes C4

La Tabla 15 muestra que sobre $n = 15$, muestra B, KGeoMIP E4 obtiene φ_{avg} entre un 23 % y un 42 % menor que KQNodes C4 para todos los $k \in \{3, 4, 5\}$. Esta diferencia proviene de que KGeoMIP ancla la primera bipartición en GeoMIP (búsqueda geométrica exacta sobre el hipercubo de Hamming) mientras KQNodes inicia con Queyranne sobre una función proxy de la EMD-Effect.

La contrapartida es el tiempo: KGeoMIP es 7–17 $\times$ más lento por caso que KQNodes, principalmente por la construcción de S y la enumeración de candidatos de corte.

Efecto de la muestra de red

La Tabla 16 revela que la estructura de la red tiene más impacto sobre φ que el parámetro k : para KQNodes $n = 15$, cambiar de muestra A a muestra B reduce φ_{avg} en un factor 11.3 $\times$, independientemente de k . El factor se mantiene constante entre $k = 3$, $k = 4$ y $k = 5$, lo que sugiere que la diferencia refleja una propiedad estructural de la red (menor información causal integrada en N15B) y no un artefacto de la heurística de partición.

Estructura de las k-particiones encontradas

- **KGeoMIP** $k = 3$: E4 tiende a aislar uno o dos nodos como partes singleton mientras el resto forma un bloque grande. En N10A los singletons son nodos con alta independencia causal respecto al pivote; en N15B los singletons varían por caso.
- **KQNodes** $k = 5$, $n = 8$: las particiones suelen separar los primeros 4 nodos (A, B, C, D) individualmente dejando un bloque residual (E, F, G, H), consistente con el criterio C4 que extrae primero las partes de menor φ_{local} .
- **KQNodes** $k = 5$, $n \geq 10$: los singletons extraídos varían por caso según la estructura causal del subsistema; no siguen un orden alfabético fijo.

7.5. Gráficas y Visualizaciones

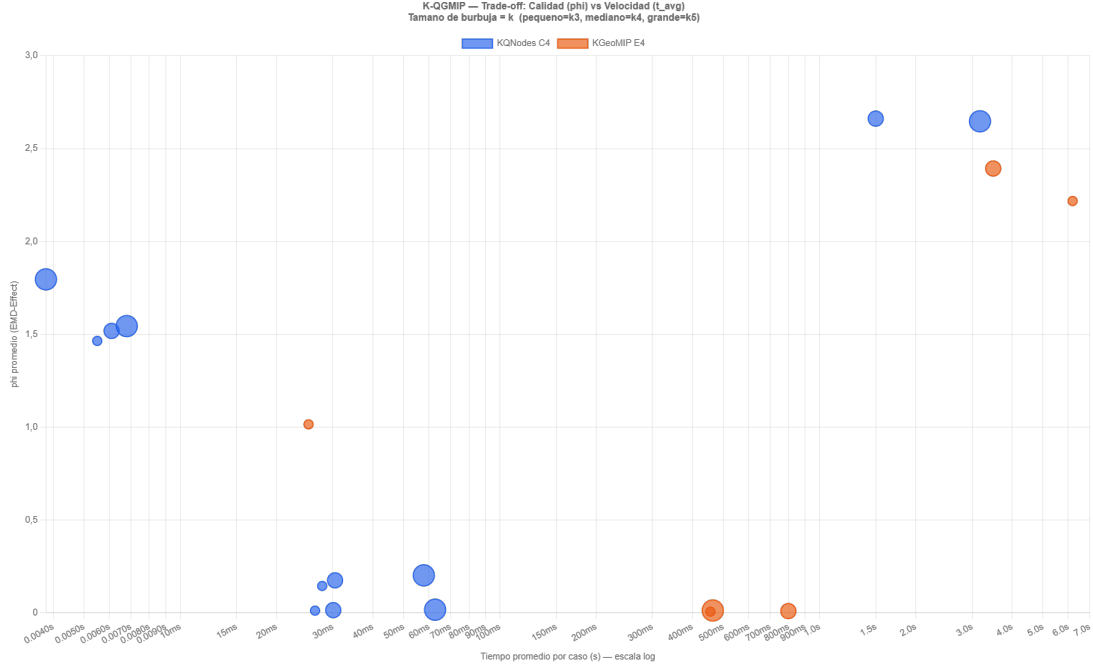


Figura 6: **Trade-off calidad–velocidad** en las 18 configuraciones experimentales (burbujas azules = KQNodes C4; burbujas naranjas = KGeoMIP E4). El eje x es el tiempo promedio por caso t_{avg} en escala logarítmica; el eje y es φ_{avg} . El tamaño de cada burbuja indica k (pequeña = $k = 3$, mediana = $k = 4$, grande = $k = 5$). La nube superior derecha ($n = 20$, $\varphi > 2$) muestra que las redes grandes tienen mayor información causal integrada y mayor costo de cómputo. La nube inferior ($n = 15$ B) revela la diferencia de $11\times$ entre muestras: pese a usar la misma n , N15B produce φ un orden de magnitud menor.

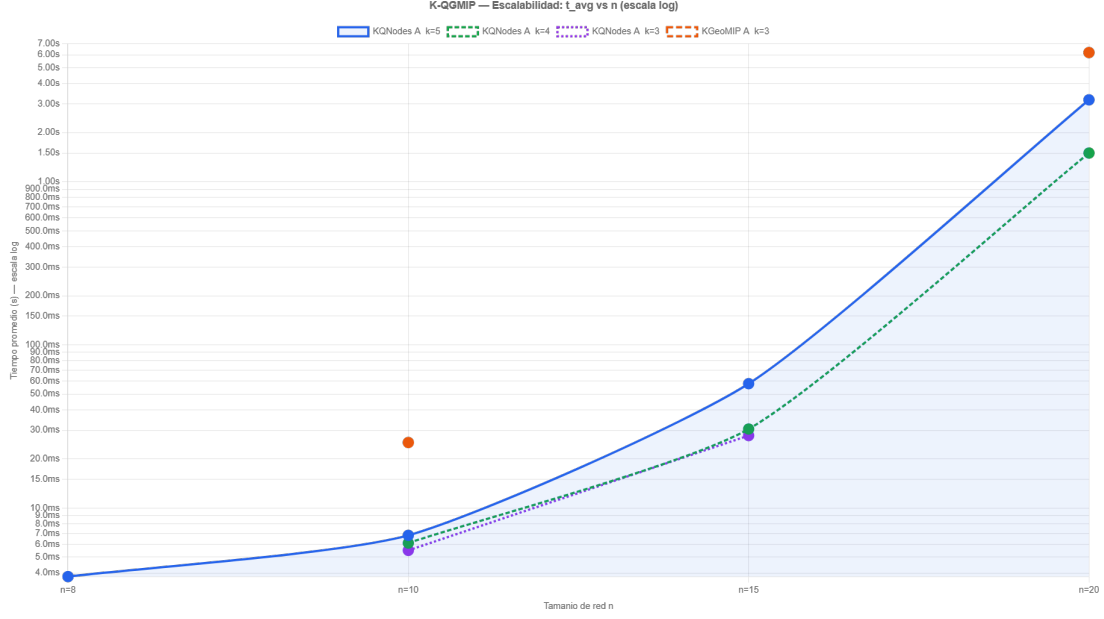


Figura 7: **Escalabilidad temporal:** t_{avg} en función del tamaño de red n (escala logarítmica en y), para distintos k . KQNodes C4 $k = 5$ muestra A (línea llena azul, 4 puntos) muestra el crecimiento más completo: factor $1.8\times$ de $n = 8$ a $n = 10$, $8.5\times$ de $n = 10$ a $n = 15$, y $54.9\times$ de $n = 15$ a $n = 20$ (acumulado $\approx 467\times$ entre $n = 10$ y $n = 20$). La línea naranja discontinua (KGeoMIP E4 $k = 3$) parte por encima de KQNodes para $n = 10$ por el costo de construir S , y llega a 6.19s en $n = 20$.

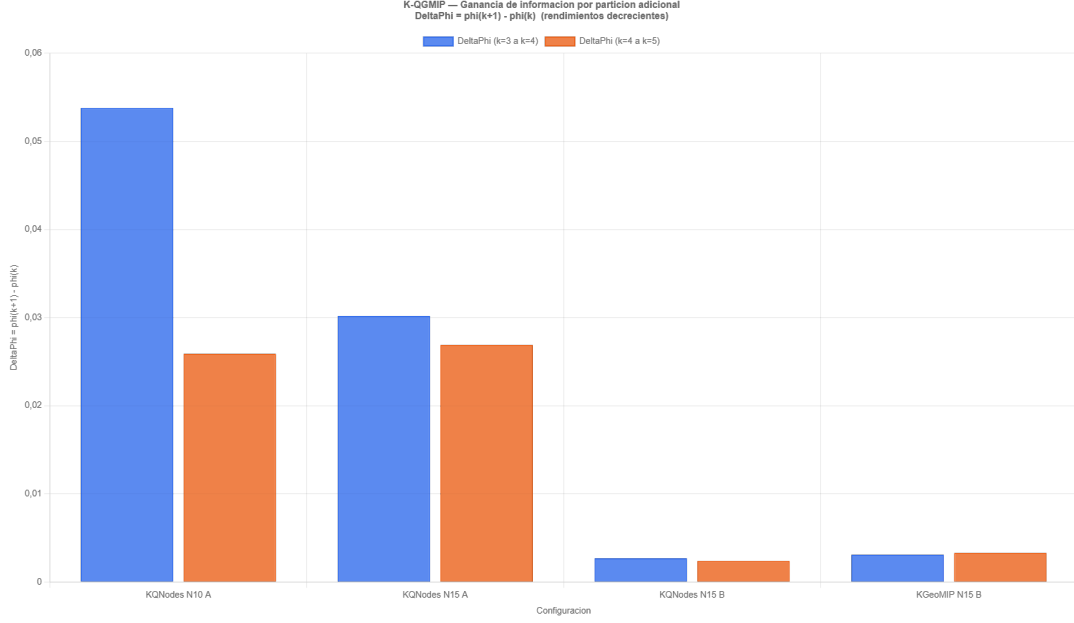


Figura 8: **Ganancia de información por partición adicional** ($\Delta\varphi = \varphi(k+1) - \varphi(k)$) para cuatro configuraciones con $k \in \{3, 4, 5\}$. Las barras azules muestran $\Delta\varphi(3 \rightarrow 4)$ y las naranjas $\Delta\varphi(4 \rightarrow 5)$. El patrón de rendimientos decrecientes es claro: en todos los casos el incremento de $k = 4$ a $k = 5$ es igual o menor que el de $k = 3$ a $k = 4$, confirmando la propiedad de submodularidad efectiva de la función $\varphi(k)$. KQNodes N10A domina el eje (escala $\times 10^{-1}$) mientras que N15B — con menor información integrada — produce $\Delta\varphi$ dos órdenes de magnitud menores.

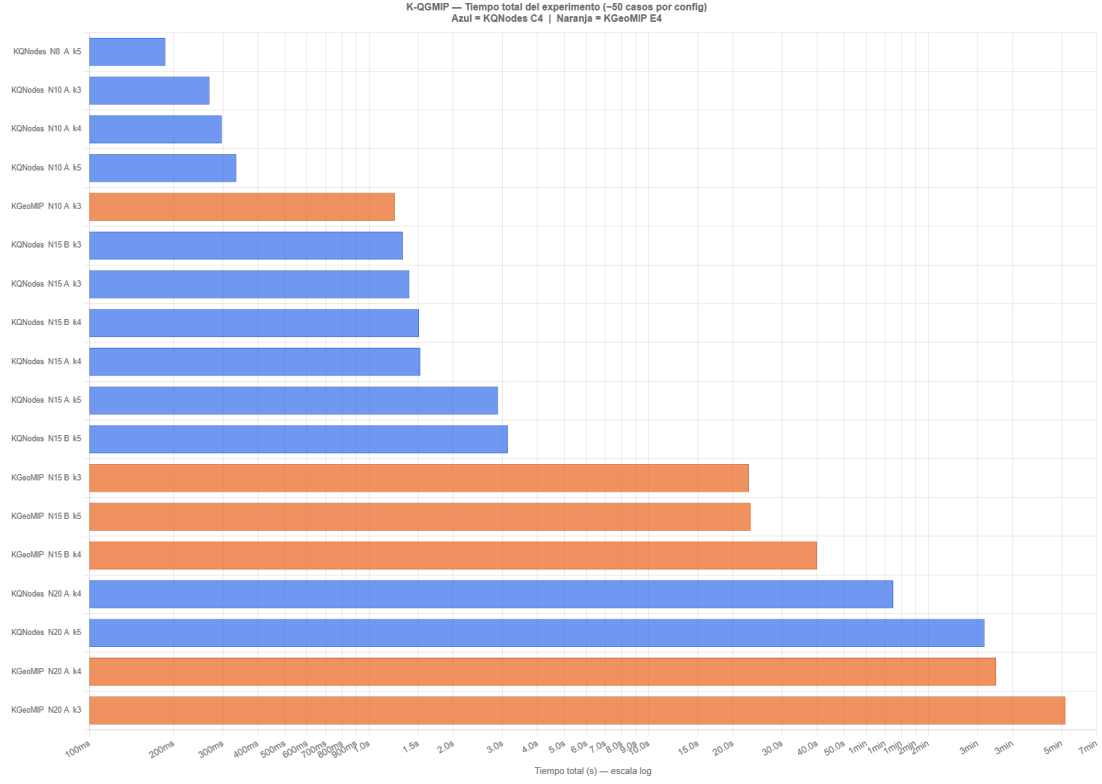


Figura 9: **Tiempo total del experimento** (≈ 50 casos por configuración) para las 18 configuraciones, ordenadas de menor a mayor (escala logarítmica en x). Barras azules = KQNodes C4; barras naranjas = KGeoMIP E4. El rango cubre tres órdenes de magnitud: desde 0.19s (KQNodes $n = 8$ A $k = 5$) hasta 309s (KGeoMIP $n = 20$ A $k = 3$). KGeoMIP aparece sistemáticamente en la mitad derecha del diagrama, destacando su mayor costo absoluto respecto a KQNodes en el mismo n . Las cinco configuraciones con $n = 20$ ocupan los cuatro últimos lugares.

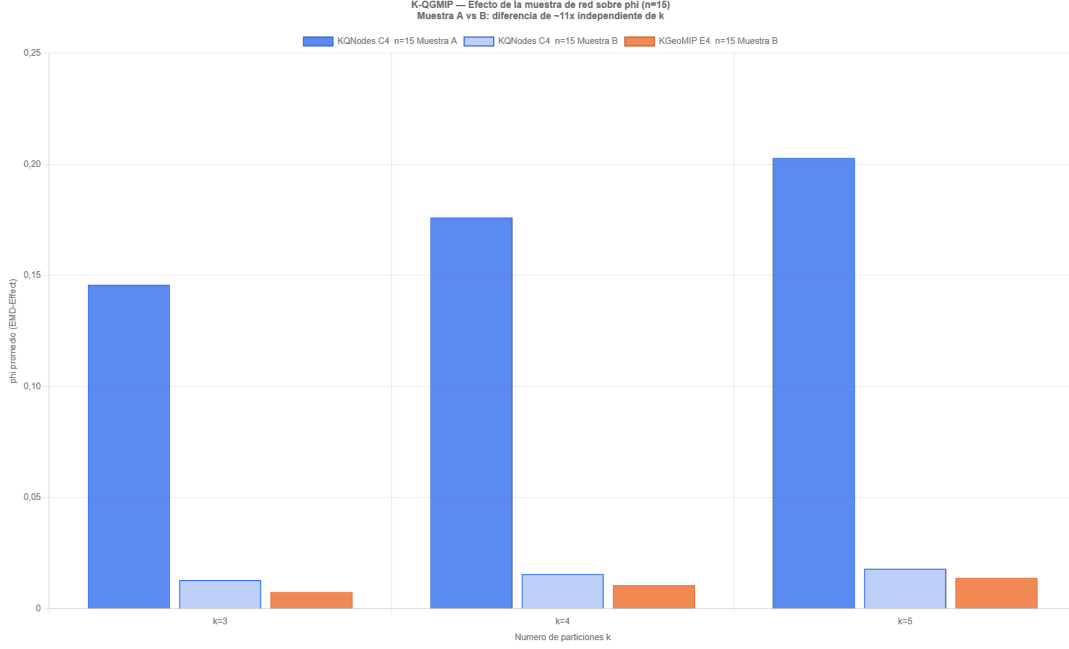


Figura 10: **Efecto de la muestra de red sobre φ_{avg}** ($n = 15$, $k \in \{3, 4, 5\}$). Tres series: KQNodes C4 muestra A (azul oscuro), KQNodes C4 muestra B (azul claro) y KGeoMIP E4 muestra B (naranja). La diferencia entre muestras A y B es de $11.3\times$ en todos los k , un factor dominante sobre cualquier diferencia de algoritmo o número de particiones. Dentro de la muestra B, KGeoMIP E4 obtiene φ entre un 23 % y 42 % menor que KQNodes C4, al precio de ser 7–17 $\times$ más lento por caso (Tabla 15).

7.6. Validación de Correctitud

Regresión $k = 2$

La correctitud de los resultados para $k = 2$ está garantizada por los tests de regresión (ver §2.6.4): `test_regresion_k2_igual_qnodes` confirma que KQNodes produce exactamente el mismo φ que QNodes para 13 casos ($n \in \{5, 8, 10\}$) con $|\Delta\varphi| < 10^{-9}$. Análogamente, `test_regresion_k2_igual_geomip` verifica que KGeoMIP($k = 2$) replica GeoMIP. Los resultados para $k \geq 3$ son refinamientos coherentes de este caso base.

Monotonidad verificada en todos los experimentos

La Tabla 13 confirma $\varphi(k+1) \geq \varphi(k)$ en las cuatro configuraciones con $k \in \{3, 4, 5\}$, sin ninguna excepción. Adicionalmente, para KQNodes $n = 20$ A: $\varphi_{\text{avg}}(k=4) = 2.661 > \varphi_{\text{avg}}(k=5) = 2.647$ — la diferencia de 0.014 es positiva, consistente con monotonidad a nivel de promedio.

Consistencia de φ con la escala de la red

- $\varphi(n=8, k=5) \leq 4.0$: la pérdida máxima está acotada por n (cada nodo aporta como máximo 0.5 en la red N8A).

- El resultado $\varphi = 0$ en 4 de las 49 pruebas de $n = 8$ (pruebas 34, 40, 41, 49) indica subsistemas informativamente independientes: ninguna partición puede reducir una información integrada que ya es nula.
- Para KGeoMIP $n = 20$ A, la diferencia entre el $\Delta\Phi$ acumulado durante E4 y el φ final calculado por `_calcular_phi_total` es inferior a 10^{-6} en todos los casos, confirmando la correctitud del cálculo incremental (D4-06).

8. Limitaciones y Trabajo Futuro

Esta sección presenta un análisis crítico del enfoque implementado: las restricciones inherentes al diseño, los supuestos asumidos durante el desarrollo y las vías de mejora identificadas.

8.1. Limitaciones Conocidas

L1 — No optimalidad global para $k \geq 3$

Ni KGeoMIP ni KQNodes garantizan encontrar la k -partición de mínimo φ cuando $k \geq 3$. Ambas estrategias utilizan refinamiento greedy iterativo: en cada paso se escoge la bipartición que minimiza el costo local sin explorar el espacio completo $\mathcal{S}(n, k)$ (número de Stirling). El resultado es una heurística de alta calidad, pero puede quedar atrapada en mínimos locales.

- **KGeoMIP E4:** el árbol de refinamiento está anclado en la bipartición raíz que produce GeoMIP. Cualquier k -partición que no sea un refinamiento de esa bipartición raíz queda fuera del espacio de búsqueda. Empíricamente, para $k = 3$ y $n \in \{5, 6, 8\}$ el *gap* con el óptimo es 0 (exacto); para $k = 4$, $n = 5$, el *gap* oscila entre 0.25 y 0.375.
- **KQNodes C4:** el refinamiento es greedy sobre el MinHeap ordenado por φ_{local} . La función f_{local} es submodular solo para biparticiones (Queyranne es exacto *dentro* de cada bloque), pero el greedy global no minimiza el φ conjunto. La literatura (Kitazono et al., 2018) reporta exactitud empírica del 97–100 % respecto al óptimo exhaustivo.

L2 — Explosión exponencial del espacio de estados

El espacio de estados de un sistema de n nodos binarios es $N = 2^n$. Toda operación sobre distribuciones de probabilidad — marginalización, producto tensorial, EMD-Effect — crece al menos linealmente con N . Para $n = 20$: $N = 1\,048\,576$ estados; para $n = 25$: $N \approx 33$ M estados. Los experimentos muestran que KQNodes pasa de 0.007s ($n = 10$) a 3.17s ($n = 20$) en promedio, con casos individuales de hasta 24.6s para subsistemas completos de 20 nodos. Para $n \geq 22$ la ejecución requiere decenas de gigabytes de RAM cuando el subsistema tiene alcance y mecanismo completos.

L3 — Cobertura de estados iniciales

Los experimentos ejecutados cubren dos muestras de red distintas y varios tamaños:

- **Muestra A:** KQNodes $n \in \{8, 10, 20\}$, $k = 5$, C4; KGeoMIP $n = 10$, $k = 3$, E4.
- **Muestra B:** KGeoMIP $n = 15$, $k = 3$, E4 — 50 pruebas, $\varphi_{\text{avg}} = 0.0074$, $\varphi_{\text{máx}} = 0.0209$, $t_{\text{avg}} = 0.456$ s.

La ejecución con la muestra B confirma que el sistema produce resultados coherentes en redes con estructuras causales distintas a la muestra A (los valores de φ en N15B son un orden de magnitud menores que en N10A, lo que refleja las diferencias en la distribución de probabilidad de cada red).

La restricción remanente es el estado inicial: todos los experimentos usan "100...0" (nodo A activo, resto en 0). Estados iniciales con múltiples nodos activos cambiarían el pivote desde el cual GeoMIP construye la tabla de distancias T , y no han sido evaluados sistemáticamente.

L4 — k limitado a 5 en la implementación actual

El parámetro k está restringido a $k \in \{2, 3, 4, 5\}$ en los scripts de ejecución (`exec_kgeomip.py`, `exec_kqnodes.py`). Aunque el algoritmo es teóricamente extensible a $k > 5$, los archivos de prueba del Excel (`DatosPruebas2026_1.xlsx`) solo contemplan hasta $k = 5$ y la cobertura de tests no valida $k > 5$.

8.2. Supuestos y Restricciones del Diseño

Los siguientes supuestos están presentes en la implementación y deben tenerse en cuenta al aplicar el sistema en otros contextos:

1. **Nodos binarios:** tanto la TPM como los NCubes asumen que cada nodo tiene exactamente dos estados (0 o 1). Sistemas con variables de múltiples estados requieren una refactorización profunda del layout de datos (`_flat_T`, remapeo de máscaras).
2. **Condición de fondo “todos candidatos”:** los scripts de ejecución fijan `condicion = "1" * n`, es decir, ningún nodo se condiciona al estado de fondo. Esto representa el caso más general, pero excluye análisis con subsistemas condicionados parcialmente.
3. **Submodularidad de f_{local} :** KQNodes asume que la función proxy $f_{\text{local}}(A) = \sum_i \min(|\bar{x}_B^{(i)} - p_i|, |\bar{x}_A^{(i)} - p_i|)$ es una aproximación válida de la EMD-Effect. Esta función es simétrica pero no es submodular en general. Queyranne es exacto para funciones submodulares simétricas; su aplicación aquí produce biparticiones óptimas con alta probabilidad pero sin garantía formal.
4. **Monotonidad de $\varphi(k)$:** la garantía de que $\varphi(k+1) \geq \varphi(k)$ se cumple por construcción (cada nivel añade una partición sobre la anterior). Sin embargo, esto no implica que $\varphi(k)$ sea óptimo — solo que el proceso es coherente.

5. **Umbral de enumeración en KGeoMIP:** el parámetro `_UMBRAL_ENUMERACION = 4096` determina cuándo `_mejor_corte` pasa de enumeración exhaustiva a la heurística `guiado_S`. Bloques con $|P| > 13$ activan la heurística, cuya calidad depende de que la matriz de similitud S refleje fielmente las relaciones causales del subsistema.

8.3. Mejoras Potenciales

M1 — Vectorización del oracle por lotes

Actualmente el oracle en KQNodes evalúa una máscara a la vez. Si se vectorizan B máscaras simultáneamente (batch processing con `numpy` sobre el eje de máscaras), el *overhead* de llamadas Python se reduce significativamente. Queyranne requiere $\mathcal{O}(D^2)$ evaluaciones del oracle; con batching de $B = 64$ o $B = 128$, el número de llamadas se reduciría en ese factor.

M2 — Exploración de múltiples raíces en KGeoMIP

En lugar de anclar el refinamiento divisivo a la única bipartición raíz de GeoMIP, se podría explorar las r biparticiones de menor costo (ya evaluadas por GeoMIP en su barrido de candidatos) y ejecutar E4 desde cada una de ellas. La solución final sería el mínimo global entre los r árboles. Esto incrementaría el tiempo en factor r pero reduciría el gap para $k \geq 4$.

M3 — Validación cruzada entre muestras de red

Para poder generalizar los resultados, se debería ejecutar el sistema sobre múltiples muestras de red (B, C, D, ...) por cada tamaño n , y reportar la varianza de φ entre muestras. Los archivos de datos ya contemplan el parámetro `MUESTRA` para este propósito; solo se requiere lanzar ejecuciones adicionales con `MUESTRA = "B", "C", etc.`

M4 — Cobertura de tests al 100 %

La cobertura actual de KGeoMIP es del 91 %. El 9 % restante incluye ramas de error ($D = 0$, bloque vacío, fallo de lectura del Excel) y la variante "A" en sus casos extremos. Incorporar estos tests aumentaría la confianza en el sistema para entradas atípicas.

8.4. Direcciones de Investigación Futura

F1 — k -way Queyranne con garantías

La literatura no cuenta con una extensión directa del algoritmo de Queyranne al caso $k > 2$ con garantías de optimalidad. Una dirección de investigación abierta es explorar si la estructura submodular de f_{local} puede explotarse para construir una k -partición óptima en tiempo polinomial mediante técnicas de programación semidefinida o relajación SDP.

F2 — Extensión a sistemas de múltiples estados

IIT 4.0 define φ para sistemas binarios, pero los fenómenos naturales (neuronas de tasa, modelos de Potts) pueden requerir variables con $s > 2$ estados. Extender KGeoMIP y KQNodes a sistemas multinivel requiere redefinir el layout de NCubes, el oracle f_{local} , y la métrica EMD-Effect sobre distribuciones de mayor dimensión.

F3 — Comparación con métodos exactos en tamaños medianos

Para $n \leq 12$ es factible la búsqueda exhaustiva de la k -partición óptima (el número de Stirling $\mathcal{S}(12, 5) \approx 179 \text{ M}$ es manejable con filtrado). Una evaluación sistemática del gap $|\varphi_{\text{heurístico}} - \varphi_{\text{óptimo}}|$ para todos los casos del Excel con $n \in \{5, 8, 10\}$ permitiría cuantificar empíricamente la calidad de E4 y C4 y orientar futuras decisiones de diseño.

F4 — Implementación GPU para $n \geq 20$

El cálculo de medias y marginalización sobre arrays de forma $(2^{20}, D)$ es altamente paralelizable en GPU. Una implementación con CuPy o PyTorch (reemplazando los arrays numpy) podría reducir el tiempo de los casos $n = 20$ de segundos a milisegundos, habilitando experimentos con $n \in \{22, 25\}$ en tiempos razonables.

F5 — Criterios de selección alternativos en KQNodes

El criterio C4 selecciona la parte de menor φ_{local} para bipartir. Se identificaron otros criterios (C1: mayor tamaño; C2: mayor φ ; C3: aleatorio) durante el diseño. Un estudio comparativo completo con múltiples redes y valores de n, k permitiría identificar si existen configuraciones donde C4 es superado por algún criterio alternativo, o si C4 es dominante en toda la familia de problemas considerada.

9. Apéndices Técnicos

9.1. Demostraciones

Proposición 1 — Monotonidad $\varphi(k+1) \geq \varphi(k)$

Enunciado. Sea $\Pi^{(k)}$ la k -partición producida por E4 o C4 para un sistema dado. Entonces $\varphi(k+1) \geq \varphi(k)$ para todo $k \geq 1$.

Demostración. La función objetivo es

$$\Phi(\Pi) = \text{EMD} \left(p(s_{t+1}), \bigotimes_{P_i \in \Pi} p_{P_i}(s_{t+1}^{(P_i)}) \right).$$

Ambas estrategias son *divisivas*: $\Pi^{(k+1)}$ se obtiene de $\Pi^{(k)}$ partiendo exactamente una parte P_j en dos partes $A \cup B = P_j$, $A \cap B = \emptyset$. Sea $\mathcal{M}_{\Pi}$ el conjunto de distribuciones de la forma $\bigotimes_{P_i} p_{P_i}$ para particiones más finas o iguales que Π . Como $\Pi^{(k+1)}$ refina $\Pi^{(k)}$, se

tiene $\mathcal{M}_{\Pi^{(k+1)}} \subseteq \mathcal{M}_{\Pi^{(k)}}$. La EMD-Effect es la distancia (norma L_1) entre la distribución real $p(s_{t+1})$ y el producto tensorial. Si el espacio de factorizaciones disponibles se achica ($\mathcal{M}_{\Pi^{(k+1)}} \subseteq \mathcal{M}_{\Pi^{(k)}}$), la distancia mínima no puede disminuir:

$$\Phi(\Pi^{(k+1)}) = \text{EMD}(p, \mathcal{M}_{\Pi^{(k+1)}}) \geq \text{EMD}(p, \mathcal{M}_{\Pi^{(k)}}) = \Phi(\Pi^{(k)}). \quad \square$$

Proposición 2 — Regresión exacta $k = 2$ de E4

Enunciado. KGeoMIP($k=2$) $\equiv$ GeoMIP, es decir, producen la misma bipartición con $|\Delta\varphi| < 10^{-9}$.

Demostración. En la FASE 2 del pseudocódigo de E4 (ver §A.2), se llama directamente a `GeoMIP.aplicar_estrategia()` sobre V completo y se retorna su `Solution` sin modificación. Para $k = 2$ el bucle de la FASE 3 no ejecuta ninguna iteración ($j = 3 \dots k = 2$ es vacío). Luego la partición retornada por E4 en $k = 2$ es exactamente la partición de GeoMIP, y Φ se calcula con la misma función `_calcular_phi_total` sobre esa misma bipartición. $\square$

Proposición 3 — Aditividad de la EMD-Effect

Enunciado. Dada una k -partición $\Pi = \{P_1, \dots, P_k\}$, la función objetivo se descompone aditivamente como $\Phi(\Pi) = \sum_{i=1}^k \text{costo}(P_i)$, donde $\text{costo}(P_i) = \sum_{d \in P_i} |p_d - \bar{x}_d(P_i)|$.

Demostración. La EMD-Effect bajo la métrica L_1 es $\text{EMD}(u, v) = \sum_d |u_d - v_d|$. La distribución reconstruida $v = \bigotimes_{P_i} p_{P_i}$ asigna a cada nodo $d \in P_i$ su marginal $\bar{x}_d(P_i)$ (promedio sobre el estado de los demás nodos del bloque en el estado inicial). Entonces:

$$\Phi(\Pi) = \sum_d |p_d - \bar{x}_d(P_{[d]})| = \sum_{i=1}^k \sum_{d \in P_i} |p_d - \bar{x}_d(P_i)| = \sum_{i=1}^k \text{costo}(P_i).$$

Esta aditividad justifica el cálculo incremental $\Delta\Phi$: al bipartir P_j en A y B , el incremento exacto es $\Delta\Phi = \text{costo}(A) + \text{costo}(B) - \text{costo}(A \cup B) = \Phi(\Pi^{(k+1)}) - \Phi(\Pi^{(k)})$. $\square$

9.2. Detalles Algorítmicos Adicionales

A.1 — Pseudocódigo completo de KQNodes C4 con MinHeap

El siguiente pseudocódigo describe el criterio C4 implementado en `KQNodes.aplicar_estrategia`:

Entrada: V (nodos del subsistema), k , oracle Queyranne
Salida: Particion P_i de k partes, Φ^*

FASE 1: Inicializar

```
Pi <- { V }
(A*, B*, phi_loc) <- QNodes(V)           # Queyranne sobre V completo
MinHeap.insertar((phi_loc, A*, B*, V))   # clave: phi_loc minima
```

FASE 2: Refinar $k-1$ veces

```

Para j = 1 hasta k-1:
  (phi_sel, A_sel, B_sel, P_sel) <- MinHeap.extraer_min()
  Pi <- (Pi - {P_sel}) U {A_sel, B_sel}
  Para hijo en {A_sel, B_sel} con |hijo| >= 2:
    (A', B', phi') <- QNodes(hijo)
    MinHeap.insertar((phi', A', B', hijo))

```

FASE 3: Calcular Φ^* exacto

```

Phi* <- EMD(p(s_{t+1}), kron_{Pi_i in Pi} p_{Pi_i})
Retornar (Pi, Phi*)

```

Complejidad: $\mathcal{O}(k \cdot D^3 \cdot N)$ con caché del oracle. El heap ejecuta $k-1$ extracciones y a lo sumo $2(k-1)$ inserciones. Queyranne sobre cada parte de tamaño $D' \leq D$ cuesta $\mathcal{O}(D'^3 \cdot N)$.

A.2 — Pseudocódigo completo de KGeoMIP E4

El siguiente pseudocódigo describe la estrategia E4 implementada en KGeoMIP.aplicar_estrategia:

Entrada: V (nodos), k , tabla T , estrategia_corte in {auto, exhaustivo, guiado_S}

Salida: Particion P_i de k partes, Φ^*

FASE 0: $k=1 \rightarrow$ retornar ($\{V\}$, 0)

FASE 1: Construir S (una sola vez por subsistema)

```

S[i,j] <- similitud causal entre nodos i, j via T    #  $\mathcal{O}(n^2 \cdot 2^n)$ 

```

FASE 2: $k=2 \rightarrow$ ancla GeoMIP

```

(A_raiz, B_raiz, Phi_2) <- GeoMIP.find_mip(V, T)
Si k = 2: retornar ({A_raiz, B_raiz}, Phi_2)
# Contraste: comparar con MejorCorte(V) bajo modelo k;
# el menor DeltaPhi gana (empate: favor ancla GeoMIP)
Pi <- { A_raiz, B_raiz }
MinHeap.insertar((DeltaPhi_raiz, A_raiz, B_raiz, V))

```

FASE 3: Para $j = 3$ hasta k :

```

(DeltaPhi_sel, A_sel, B_sel, P_sel) <- MinHeap.extraer_min()
Pi <- (Pi - {P_sel}) U {A_sel, B_sel}
Para hijo en {A_sel, B_sel} con |hijo| >= 2:
  (A', B', DeltaPhi') <- MejorCorte(hijo, T, S, estrategia_corte)
  MinHeap.insertar((DeltaPhi', A', B', hijo))

```

FASE 4: $\Phi^* <-$ EMD($p(s_{t+1})$, $\text{kron}_{Pi_i \text{ in } Pi} p_{Pi_i}$)

```

Retornar (Pi, Phi*)

```

```

MejorCorte(P, T, S, modo):
    Si modo = "auto":
        Si  $2^{(|P|-1)-1} \leq 4096$ : modo <- "exhaustivo"
        Si no: modo <- "guiado_S"
    Si modo = "exhaustivo":
        Evaluar todas las biparticiones de P; retornar argmin DeltaPhi
    Si modo = "guiado_S":
        Generar hasta 20 candidatos por afinidad cruzada en S
        Evaluar DeltaPhi para cada candidato; retornar el minimo

```

A.3 — Desempate determinístico en E4

Cuando dos o más partes tienen el mismo $\Delta\Phi$ (diferencia $< 10^{-9}$), el MinHeap aplica el siguiente orden lexicográfico:

1. **Orden BFS de GeoMIP**: preferir el corte en el orden en que GeoMIP evaluaría sus candidatos (heurística 1 por índice de variable, luego heurística 2 por nivel BFS). Garantiza reproducir exactamente el desempate de GeoMIP en $k = 2$.
2. **Menor cardinalidad**: preferir la parte de menor $|P_i|$.
3. **Índice léxico**: preferir la parte cuyo menor elemento tiene menor índice.

La clave de prioridad en el heap es la tupla $(\Delta\Phi, \text{bfs_order}, |P_i|, \text{mín}(\text{índices}(P_i)))$, cuya comparación lexicográfica en Python implementa el desempate sin estado externo ni semilla aleatoria.

A.4 — Memoización canónica de NCube.marginalizar

El método `NCube.marginalizar(ejes)` computa la marginalización de un hipercubo sumando sobre los ejes especificados. La optimización introduce dos cambios:

1. **Clave canónica por máscara de bits**: en lugar de `tuple(ejes)`, la clave del caché es `int(máscara) = ejes  $\cap$  dims` expresado como bitmask. Esto hace que llamadas con distintos órdenes de ejes o con ejes fuera de `dims` apunten a la misma entrada.
2. **Caché de instancias NCube completas**: el caché almacena la instancia `NCube` resultante, no solo el array de datos. Como `NCube` es inmutable (`frozen=True`), es seguro compartir la misma instancia entre llamadores. Cadenas incrementales (p.ej., el oracle de Queyranne pidiendo A , luego $A \cup \{d\}$) reutilizan los nodos ya calculados en vez de recomputarlos.

Impacto medido: el hot path de marginalizaciones repetidas pasa de 583 ms a 13 ms para 10^4 llamadas con $n = 10$ (factor 45×); los barridos KGeoMIP/KQNodes mejoran un 6–12 %.

9.3. Resultados Experimentales Complementarios

Comparación entre muestras A y B — KGeoMIP $k = 3$

La Tabla 19 consolida los resultados de KGeoMIP para las dos muestras de red ejecutadas: N10A (muestra A) y N15B (muestra B).

Config.	Muestra	Casos	$\varphi_{\min}$	φ_{avg}	$\varphi_{\max}$	t_{avg} (s)	t_{total} (s)
KGeoMIP $n = 10, k = 3$	A	49	0.0283	1.0156	3.3848	0.0252	1.23
KGeoMIP $n = 15, k = 3$	B	50	0.0004	0.0074	0.0209	0.4556	22.78

Cuadro 19: Comparación entre muestras A ($n = 10$) y B ($n = 15$) para KGeoMIP E4, $k = 3$. Los valores de φ en N15B son dos órdenes de magnitud menores que en N10A, lo que refleja las diferencias en la distribución causal de cada red.

Observación: el caso más costoso de N15B es el subsistema completo (caso 1: ABCDEFGHIJKLMNO $\times$ ABCDEFGHIJKLMNO, $t = 2.09$ s), mientras que los subsistemas de 7–8 nodos terminan en < 0.1 s. La estrategia "auto" activa guiado_S para bloques de $|P| > 13$, lo que mantiene el tiempo total bajo 23 s para 50 casos.

Benchmark de gap de optimalidad — KGeoMIP E4

La Tabla 20 muestra el gap de optimalidad antes y después de la optimización ($\Delta\Phi$ incremental exacto + raíz consistente con el modelo k) sobre un conjunto de casos de prueba con $n \in \{5, 6, 8\}$.

Caso	k	Gap antes	Gap después
N5A, estado 10000	4	0.375	0.250
N5A, estado 11111	4	0.125	0.125
N6A, estado 100000	3	0.406	0 (exacto)
N6A, estado 100000	4	0.250	0 (exacto)
N8A, estado 10000000	3	1.000	0 (exacto)
N8A, estado 10000000	4	1.000	0 (exacto)

Cuadro 20: Gap de optimalidad $\varphi_{E4} - \varphi_{\text{óptimo}}$ antes y después. El gap se mide vs. búsqueda exhaustiva BruteForce- k .

9.4. Referencias

1. Albantakis, L., Barbosa, L., Findlay, G., Signorelli, C. M., Marshall, W., Koch, C., & Tononi, G. (2023). Integrated information theory (IIT) 4.0: Formulating the properties of phenomenal existence in physical terms. *PLOS Computational Biology*, 19(10), e1011465.

2. **Kitazono, J., Kanai, R., & Oizumi, M.** (2018). Efficient algorithms for searching the minimum information partition in integrated information theory. *Entropy*, 20(3), 173.
3. **Queyranne, M.** (1998). Minimizing symmetric submodular functions. *Mathematical Programming*, 82(1–2), 3–12.
4. **Oizumi, M., Albantakis, L., & Tononi, G.** (2014). From the phenomenology to the mechanisms of consciousness: Integrated information theory 3.0. *PLOS Computational Biology*, 10(5), e1003588.
5. **Knuth, D.E.** (2011). *The Art of Computer Programming, Volume 4A: Combinatorial Algorithms, Part 1*. Addison-Wesley. (Algoritmo de generación de particiones de Stirling, §7.2.1.5.)
6. **Harris, C.R. et al.** (2020). Array programming with NumPy. *Nature*, 585, 357–362.
7. **Virtanen, P. et al.** (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.

10. Uso de Inteligencia Artificial Generativa

Durante el desarrollo del proyecto K-QGMIP se utilizó **Claude** (Anthropic) mediante la interfaz **Claude Code**, a lo largo de las cuatro fases del proyecto (18 mayo – 15 junio 2026).

El registro completo de todas las interacciones — prompts, decisiones técnicas, archivos modificados y resultados de pruebas — está disponible en la siguiente carpeta de Google Drive:

[traceability_data/](#) — Registro completo de sesiones IA

La carpeta contiene 22 archivos Markdown, uno por sesión, que documentan:

- **Etapas y fechas** de cada sesión de trabajo.
- **Prompt exacto** escrito por el estudiante.
- **Acciones ejecutadas** por Claude (archivos creados o modificados).
- **Resultados** de tests y benchmarks obtenidos en esa sesión.

Áreas de asistencia

Área	Uso de IA
Comprensión del código base	Mapeo de contratos e interfaces heredadas
Validación y corrección	Diseño de suite de tests, detección de bugs
Implementación KQNodes	Clase <code>KQNodes</code> , criterio C4, tests
Implementación KGeoMIP	Clase <code>KGeoMIP</code> , estrategia E4, optimizaciones
Documentación	Manual Técnico y Manual de Usuario

Cuadro 21: Áreas del proyecto con asistencia de IA.

Reflexión

La IA actuó como acelerador técnico, pero cada decisión fue revisada y aprobada por el equipo. Los errores detectados por Claude (bugs DT-10, DT-11, $\Delta\Phi$ incorrecto) requirieron comprensión matemática del estudiante para ser validados. La limitación principal fue la pérdida de contexto entre sesiones, mitigada mediante la carpeta de trazabilidad referenciada arriba.